学术AI鉴伪系统概要设计说明书（迭代版）

(AID-SD-1)

概要设计说明书

<table>
<tr><td rowspan="4">文件状态：
[✓] 草稿
[ ] 正式发布
[ ] 正在修改</td><td>文件标识：</td><td>AID-SD-1</td></tr>
<tr><td>当前版本：</td><td>v0.1</td></tr>
<tr><td>作　　者：</td><td>刘宇童　聂薪羽　苏天
翟润　张与耕</td></tr>
<tr><td>完成日期：</td><td>2026 年 4 月 17 日</td></tr>
</table>

目录

1 引言 **7**
1.1 编写目的 7
1.2 文档范围 7
1.3 参考资料 7
1.4 术语说明 7

2 系统概述 **7**
2.1 设计背景 7
2.2 现有系统基础 8
2.3 现有系统与迭代目标差异 8
2.4 设计目标 9
2.5 用户与角色视角 9
2.6 业务范围 10
2.7 设计原则 10
2.8 系统边界 10

3 总体架构设计 **11**
3.1 总体架构说明 11
3.2 架构选型依据 11
3.3 系统总体静态结构类图 12
3.4 统一资源与检测扩展关系类图 13
3.5 系统组成 13
3.6 子系统职责划分 14
3.7 核心调用链设计 14
3.8 分层设计 15
3.9 架构约束与关键设计决策 15
3.10 面向迭代的架构调整方向 16
3.11 关键架构优势 16
3.12 总体架构小结 16

4 模块设计 **16**
4.1 模块划分原则 16
4.2 模块总体关系类图 17
4.3 模块总览 17
4.4 用户与权限模块（SD-YHQX-0001） 18
4.4.1 模块目标 18
4.4.2 模块职责 18
4.4.3 功能组成 19
4.4.4 输入与输出 19
4.4.5 核心数据对象 19
4.4.6 处理流程 19
4.4.7 模块接口与交互关系 20

4.4.8 设计要点 20
4.4.9 后续扩展方向 20
4.5 组织与成员模块（SD-ZZCY-0001） 20
4.5.1 模块目标 20
4.5.2 模块职责 20
4.5.3 功能组成 21
4.5.4 输入与输出 21
4.5.5 核心数据对象 21
4.5.6 处理流程 21
4.5.7 模块接口与交互关系 22
4.5.8 设计要点 22
4.5.9 后续扩展方向 22
4.6 资源管理模块（SD-ZYGL-0001） 22
4.6.1 模块目标 22
4.6.2 模块职责 22
4.6.3 功能组成 23
4.6.4 输入与输出 23
4.6.5 核心数据对象 23
4.6.6 处理流程 23
4.6.7 模块接口与交互关系 23
4.6.8 设计要点 23
4.6.9 后续扩展方向 24
4.7 检测任务管理模块（SD-JCRW-0001） 24
4.7.1 模块目标 24
4.7.2 模块职责 24
4.7.3 功能组成 24
4.7.4 输入与输出 24
4.7.5 核心数据对象 24
4.7.6 处理流程 25
4.7.7 模块接口与交互关系 25
4.7.8 设计要点 25
4.7.9 后续扩展方向 25
4.8 图像检测模块（SD-TPJC-0001） 25
4.8.1 模块目标 25
4.8.2 模块职责 25
4.8.3 功能组成 26
4.8.4 输入与输出 26
4.8.5 核心数据对象 26
4.8.6 处理流程 26
4.8.7 模块接口与交互关系 27
4.8.8 设计要点 27
4.8.9 后续扩展方向 27
4.9 论文检测模块（SD-LWJC-0001） 27

4.9.1 模块目标 27
4.9.2 模块职责 27
4.9.3 功能组成 28
4.9.4 输入与输出 28
4.9.5 核心数据对象 28
4.9.6 处理流程 28
4.9.7 模块接口与交互关系 29
4.9.8 设计要点 29
4.9.9 后续扩展方向 29
4.10 评审检测模块（SD-PSJC-0001） 29
4.10.1 模块目标 29
4.10.2 模块职责 29
4.10.3 功能组成 30
4.10.4 输入与输出 30
4.10.5 核心数据对象 30
4.10.6 处理流程 30
4.10.7 模块接口与交互关系 31
4.10.8 设计要点 31
4.10.9 后续扩展方向 31
4.11 人工审核模块（SD-RGSH-0001） 31
4.11.1 模块目标 31
4.11.2 模块职责 31
4.11.3 功能组成 32
4.11.4 输入与输出 32
4.11.5 核心数据对象 32
4.11.6 处理流程 32
4.11.7 模块接口与交互关系 32
4.11.8 设计要点 32
4.11.9 后续扩展方向 33
4.12 报告服务模块（SD-BGFW-0001） 33
4.12.1 模块目标 33
4.12.2 模块职责 33
4.12.3 功能组成 33
4.12.4 输入与输出 33
4.12.5 核心数据对象 33
4.12.6 处理流程 34
4.12.7 模块接口与交互关系 34
4.12.8 设计要点 34
4.12.9 后续扩展方向 34
4.13 通知消息模块（SD-TZXX-0001） 34
4.13.1 模块目标 34
4.13.2 模块职责 34
4.13.3 功能组成 34

4.13.4 输入与输出 35
4.13.5 核心数据对象 35
4.13.6 处理流程 35
4.13.7 模块接口与交互关系 35
4.13.8 设计要点 35
4.13.9 后续扩展方向 36
4.14 统计日志模块（SD-TJRZ-0001） 36
4.14.1 模块目标 36
4.14.2 模块职责 36
4.14.3 功能组成 36
4.14.4 输入与输出 36
4.14.5 核心数据对象 36
4.14.6 处理流程 37
4.14.7 模块接口与交互关系 37
4.14.8 设计要点 37
4.14.9 后续扩展方向 37
4.15 本章小结 37

5 数据设计 37
5.1 数据设计目标 37
5.2 数据设计原则 38
5.3 核心数据模型类图 39
5.4 核心实体总览 39
5.5 核心实体说明 40
5.5.1 用户与组织域 40
5.5.2 资源与任务域 40
5.5.3 结果与审核域 40
5.5.4 治理与协同域 41
5.6 实体关系说明 41
5.7 关键数据项设计 41
5.8 状态设计 42
5.9 数据流转设计 42
5.10 完整性与一致性约束 42
5.11 索引与查询设计 43
5.12 数据安全与归档设计 43
5.13 兼容与演进说明 43
5.14 本章小结 43

6 接口设计 44
6.1 接口设计目标 44
6.2 接口设计原则 44
6.3 接口总体分层 44
6.4 外部接口分类 45

6.5 内部接口分类 45
6.6 统一接口约定 45
6.7 当前项目外部接口总览 46
6.8 用户认证与个人信息接口 46
6.9 组织与成员接口 46
6.10 资源上传与资源管理接口 47
6.11 检测任务与检测结果接口 47
6.12 人工审核接口 48
6.13 通知与实时消息接口 49
6.14 管理端治理与统计接口 49
6.15 内部接口设计 50
6.15.1 后端与异步任务系统接口 50
6.15.2 后端与 AI 推理服务接口 50
6.15.3 后端与文件存储接口 51
6.15.4 后端与消息通道接口 51
6.16 接口安全设计 51
6.17 接口演进方向 51
6.18 本章小结 51

7 关键流程设计 52
7.1 资源上传与检测流程 52
7.1.1 流程输入 52
7.1.2 处理步骤 52
7.1.3 流程输出 53
7.1.4 关键控制点 53
7.2 图像检测流程 53
7.2.1 流程阶段划分 53
7.2.2 详细处理步骤 53
7.2.3 流程输出 54
7.2.4 异常与恢复 54
7.3 论文检测流程 54
7.3.1 目标流程 54
7.3.2 与当前实现的衔接点 54
7.3.3 设计关注点 54
7.4 Review检测流程 55
7.4.1 目标流程 55
7.4.2 与当前实现的衔接点 55
7.4.3 设计关注点 55
7.5 人工审核流程 55
7.5.1 处理步骤 56
7.5.2 流程输出 56
7.5.3 关键控制点 56
7.6 通知与状态同步流程 56

7.6.1 处理步骤 57
7.6.2 流程特点 57
7.7 流程图说明 57

8 运行与部署设计 57
8.1 部署结构 57
8.2 运行机制 58
8.3 部署约束 58

9 质量属性设计 58
9.1 性能设计 58
9.2 安全设计 58
9.3 可维护性设计 58
9.4 可扩展性设计 58
9.5 可靠性设计 58

10 设计约束与待定问题 59
10.1 当前约束 59
10.2 待定问题 59
10.3 后续工作 59

1 引言

1.1 编写目的

本文档用于描述学术AI鉴伪系统在迭代开发阶段的概要设计方案，明确系统的总体结构、模块划分、关键数据关系、主要交互流程以及部署与质量设计，为后续详细设计、编码实现、测试验证和系统演进提供依据。

1.2 文档范围

本文档重点覆盖以下内容：

- 迭代后系统的总体架构与模块边界；
- 用户端、管理端、后端服务与AI推理服务之间的协作关系；
- 学术图像检测、全篇论文检测、Review检测、人工审核与报告生成的设计方案；
- 核心数据实体、接口分层、关键业务流程与运行部署设计；
- 可维护性、可扩展性、安全性与性能等质量属性设计。

1.3 参考资料

- 《学术AI鉴伪系统需求规格说明书（迭代版）》；
- 当前项目代码仓库中的前端、后端与AI服务实现；
- 项目现有接口文档、实体表与交接说明。

1.4 术语说明

术语	说明
学术资源	系统支持处理的业务对象，包括学术图像、论文文件与同行评审Review。
检测任务	用户针对一个或多个学术资源发起的一次检测处理请求。
检测结果	系统对指定资源执行自动检测后产生的结构化输出。
人工审核	在自动检测基础上，由人工参与复核、判断或补充说明的业务过程。
综合报告	汇总自动检测与人工审核信息后形成的可下载输出结果。

2 系统概述

2.1 设计背景

学术内容真实性识别已不再局限于传统图像篡改检测场景。随着生成式人工智能技术在论文写作、图片生成与学术评审文本中的广泛使用，学术不端风险逐渐呈现出多类型、跨环节与高隐蔽性的特点。对于期刊、科研机构和研究团队而言，仅依靠单一内容类型、单一检测模型或人工抽查方式，已难以满足对学术资源进行系统化风险识别的需求。

当前项目已完成一轮以学术图像检测为核心的系统建设，具备用户注册登录、图像上传、图片提取、图像自动检测、人工审核、结果通知以及管理端统计等基础能力。从代码实现上看，系统已经形成了由用户端、管理端、Django后端、异步任务系统和独立AI推理服务组成的可运行基础框架，为后续迭代提供了较好的工程基础。

然而，现有系统的业务抽象和数据模型仍明显围绕图像检测链路展开，尚不能完整支撑迭代需求中提出的全篇论文检测、Review检测、综合报告统一生成、模型配置治理以及论文/Review专项日志管理等能力。因此，本轮概要设计的核心任务不是简单补充页面或接口，而是在保留现有可用能力的基础上，完成系统从“图像检测平台”向“综合学术AI鉴伪平台”的结构升级。

2.2 现有系统基础

结合当前项目代码，现有系统已经具备如下实现基础：

- 在前端层面，已分别实现用户端与管理端两套Web应用，能够支持基本业务操作、历史记录查询、人工审核入口与管理统计展示；
- 在后端层面，已形成以Django为核心的业务服务，提供用户认证、资源上传、检测任务管理、人工审核、组织管理、通知与管理端接口；
- 在异步处理层面，已集成Celery与Redis，支持检测任务异步执行、批处理调度、任务状态更新与结果回写；
- 在AI推理层面，已具备独立的图像推理服务和图像检测流水线，能够执行图像伪造识别、可疑区域生成与结果封装；
- 在数据层面，已具备用户、组织、上传文件、检测任务、检测结果、人工审核、日志与通知等核心表结构。

上述基础说明，当前系统并非从零开始设计，而是在已有实现上进行面向迭代需求的重构与扩展。因此，概要设计需要同时考虑两个方面：一方面要充分复用已有实现中成熟的模块和机制；另一方面要识别现有设计中图像中心化、模型耦合和实体抽象不足等问题，并提出统一化、可扩展的改造方案。

2.3 现有系统与迭代目标差异

从需求文档与现有实现对照来看，当前系统与本轮目标系统之间主要存在以下差异：

对比项	当前系统现状	本轮迭代目标
核心检测对象	以学术图像为主	统一支持学术图像、全篇论文与Review
业务抽象中心	以图像上传和图像结果为中心	以统一学术资源、统一检测任务和统一报告为中心
AI能力组织方式	图像检测链路已落地，模型调用与业务耦合较强	不同检测对象对应不同检测适配器，模型调用更加配置化与模块化
结果输出方式	以图像检测结果和图像报告为主	统一支持图像、论文、Review结果展示与综合报告生成

人工审核范围	以图像审核流程为主	需要与更广义的综合鉴伪流程进行衔接
日志与治理能力	已具备操作日志和部分统计能力	进一步增加论文日志、Review日志、模型配置和资源治理能力

因此，本轮概要设计的重点不在于重写已有图像检测流程本身，而在于建立能够承载三类学术资源的总体结构，使现有功能能够作为其中一个可复用子链路纳入统一架构。

2.4 设计目标

本轮概要设计的总体目标如下：

- 在保留现有学术图像检测能力的前提下，将系统扩展为面向学术图像、全篇论文和Review的统一鉴伪平台；
- 对上传、检测、审核、报告、日志与后台治理形成一致的系统结构，避免不同资源类型各自独立演化；
- 降低现有实现中图像中心化带来的耦合问题，为新增资源类型与新增检测模型预留稳定扩展点；
- 明确前端、后端、异步任务系统、AI推理服务、数据库和文件存储之间的职责边界与协作关系；
- 为后续详细设计、接口扩展、数据库调整和模块重构提供统一依据。

进一步地，从工程交付角度看，本轮设计还应满足以下目标：

- 能够支撑逐步迭代开发，而不是要求一次性完成全部重构；
- 能够兼容已有图像检测代码与数据结构，降低重构风险；
- 能够将新增论文检测和Review检测纳入现有任务调度、通知和管理体系中；
- 能够支撑课程项目背景下的文档撰写、演示展示和后续验收。

2.5 用户与角色视角

根据需求文档和当前实现，本系统主要服务于以下三类角色：

- **普通用户**：主要包括科研人员、学生、编辑和其他资源提交者。其关注点在于上传资源、发起检测、查看结果、下载报告以及申请人工审核；
- **审核参与者**：主要指具有人工审核职责的用户。其关注点在于接收审核任务、查看资源与自动检测结果、提交审核结论及原因说明；
- **管理员**：包括组织管理员和超级管理员。其关注点在于组织和用户治理、审核审批、资源管理、日志查询、统计分析和模型配置。

不同角色共享同一业务平台，但其功能入口、数据访问范围和操作边界不同。因此，概要设计中需要明确角色分层，而不能将系统视为单一用户形态的应用。

2.6 业务范围

结合需求规格说明书，本轮迭代后系统的业务范围主要包括以下几个部分：

- **资源接入范围**：系统支持图像文件、论文文件和Review文本三类资源进入统一检测流程；
- **自动检测范围**：系统支持对图像伪造风险、论文AIGC风险和Review模板化/AI生成倾向进行自动分析；
- **结果输出范围**：系统支持检测结果展示、结构化结果查询与综合报告生成；
- **人工复核范围**：系统支持用户发起人工审核请求，并在必要时引入审核参与者和管理员共同完成复核流程；
- **后台治理范围**：系统支持对用户、组织、资源、日志、审核和模型进行统一管理。

2.7 设计原则

为了保证本轮系统设计既能兼容当前实现、又能支撑后续演进，概要设计遵循以下原则：

- **模块化原则**：按业务职责划分模块，减少不同模块之间的直接耦合；
- **统一抽象原则**：对不同类型学术资源采用统一任务与结果抽象，不再仅围绕图像对象组织系统；
- **异步处理原则**：长时检测任务通过异步机制执行，避免阻塞用户交互主流程；
- **可扩展原则**：为新增检测对象、检测模型和管理能力预留扩展接口；
- **可追溯原则**：关键业务过程保留任务状态、日志与报告映射关系，保证结果可回查；
- **兼容演进原则**：在设计上兼顾现有系统可复用部分与目标结构升级，避免一次性推翻现有实现。

2.8 系统边界

从系统边界看，本系统主要由用户端、管理端、后端业务服务、异步任务处理服务、AI推理服务、数据库和文件存储构成。系统对外提供Web访问与业务接口能力，对内通过HTTP调用、消息队列、数据库记录和文件共享完成各组件之间的协作。

系统边界之外的内容主要包括：

- 具体检测模型的训练过程与训练平台；
- 第三方学术数据库或外部内容平台的直接接入；
- 大规模生产级运维体系中的弹性调度与云原生编排能力；
- 不属于本课程项目实现范围的深度审计或合规系统。

也就是说，本文档关注的是系统如何组织、调用和管理检测能力，而不是对底层模型训练机制或外部科研生态系统做全面设计。

3 总体架构设计

3.1 总体架构说明

系统总体上采用“前后端分离 + 统一业务服务 + 异步任务调度 + 独立AI推理服务”的组合架构。该架构不是脱离现有工程基础的重新假设，而是在当前项目代码已经形成用户端、管理端、Django 后端、Celery 异步任务、Redis 消息中转和独立图像推理服务的基础上，对未来迭代目标进行整理和提升后的结果。

从业务演进方向看，系统已经不再只是处理图像伪造检测，而是需要统一承载学术图像、全篇论文和 Review 文本三类检测对象。因此，体系结构必须从“围绕图像功能组织实现”过渡为“围绕统一资源、统一任务和统一结果组织实现”。前端负责交互与展示，后端负责业务编排，异步任务系统负责长耗时检测任务的排队和执行，AI 推理服务负责具体模型运行，数据库与文件存储负责持久化和追溯。通过这种职责拆分，可以在保留现有图像链路的同时，为论文检测、Review 检测和综合报告生成预留稳定扩展点。

需要说明的是，本章为了强调系统静态结构和依赖关系，统一采用 UML 类图风格描述主要子系统和核心抽象，而不使用传统流程框图。这里的“类”并不局限于面向对象代码类，而是作为对系统中稳定职责单元的结构化表达。

3.2 架构选型依据

本系统选择当前总体架构，主要基于以下考虑：

- **Web 访问是主要交互方式**。系统主要面向浏览器端用户，用户端和管理端存在明显不同的页面需求与权限边界，采用前后端分离结构更有利于界面独立演进；
- **检测任务执行时间较长**。无论是图像检测、论文检测还是 Review 检测，都可能涉及文件解析、特征提取、批量切分和模型推理，不适合同步阻塞式请求处理；
- **检测对象存在明显差异**。图像、论文和 Review 的输入格式、预处理方式、结果结构和模型调用方式并不一致，需要通过统一抽象之上的适配机制来隔离差异；
- **系统需要持续迭代扩展**。本轮只是综合学术 AI 鉴伪平台的第一阶段扩展，后续仍可能增加模型管理、报告策略、组织治理和更细粒度的人工复核能力，因此架构必须支持增量演进；
- **当前代码已具备可复用基础**。继续整理和提升现有 Django、Celery、Redis、WebSocket 与独立 AI 服务体系，相比完全推翻重写更具工程可行性。

3.3 系统总体静态结构类图

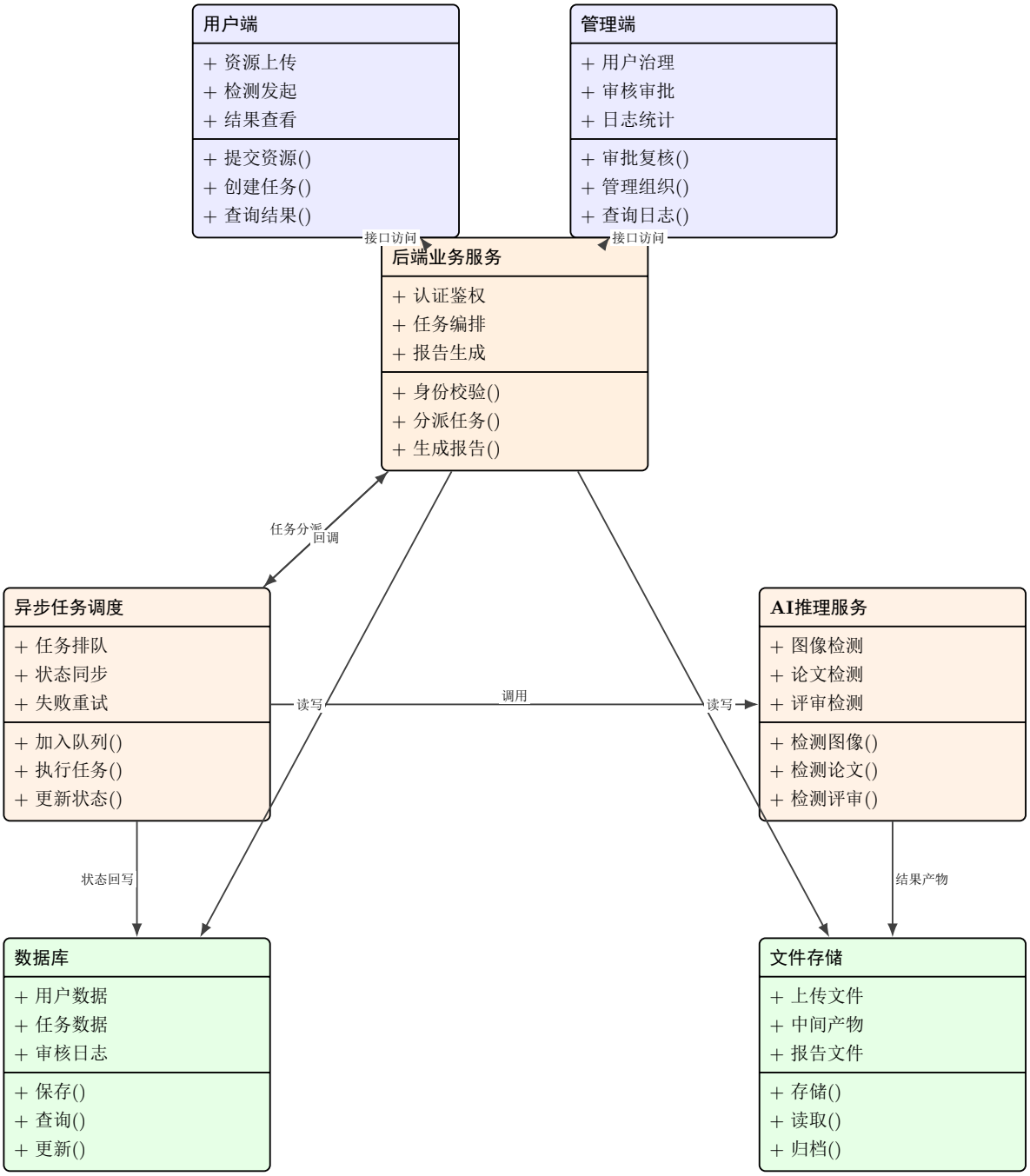

图 1: 系统总体静态结构 UML 类图

图 3-1 从系统静态依赖关系角度展示了本项目的总体结构。用户端和管理端分别面向普通用户与管理用户，通过统一后端业务服务访问平台能力；后端业务服务一方面直接管理认证鉴权、组织与资源治理、结果查询和报告生成等同步能力，另一方面通过异步任务调度子系统将长耗时检测请求提交到 AI 推理服务；推理结果再由业务服务统一汇总并落入数据库与文件存储，最终由通知与状态反馈机制返回到前端界面。

与传统的分层框图相比，使用类图样式表达的好处在于可以更清楚地表示各个稳定子系统之间的静态关联关系、依赖方向和职责边界，便于后续详细设计时继续向模块类图和实体类图细化。

3.4 统一资源与检测扩展关系类图

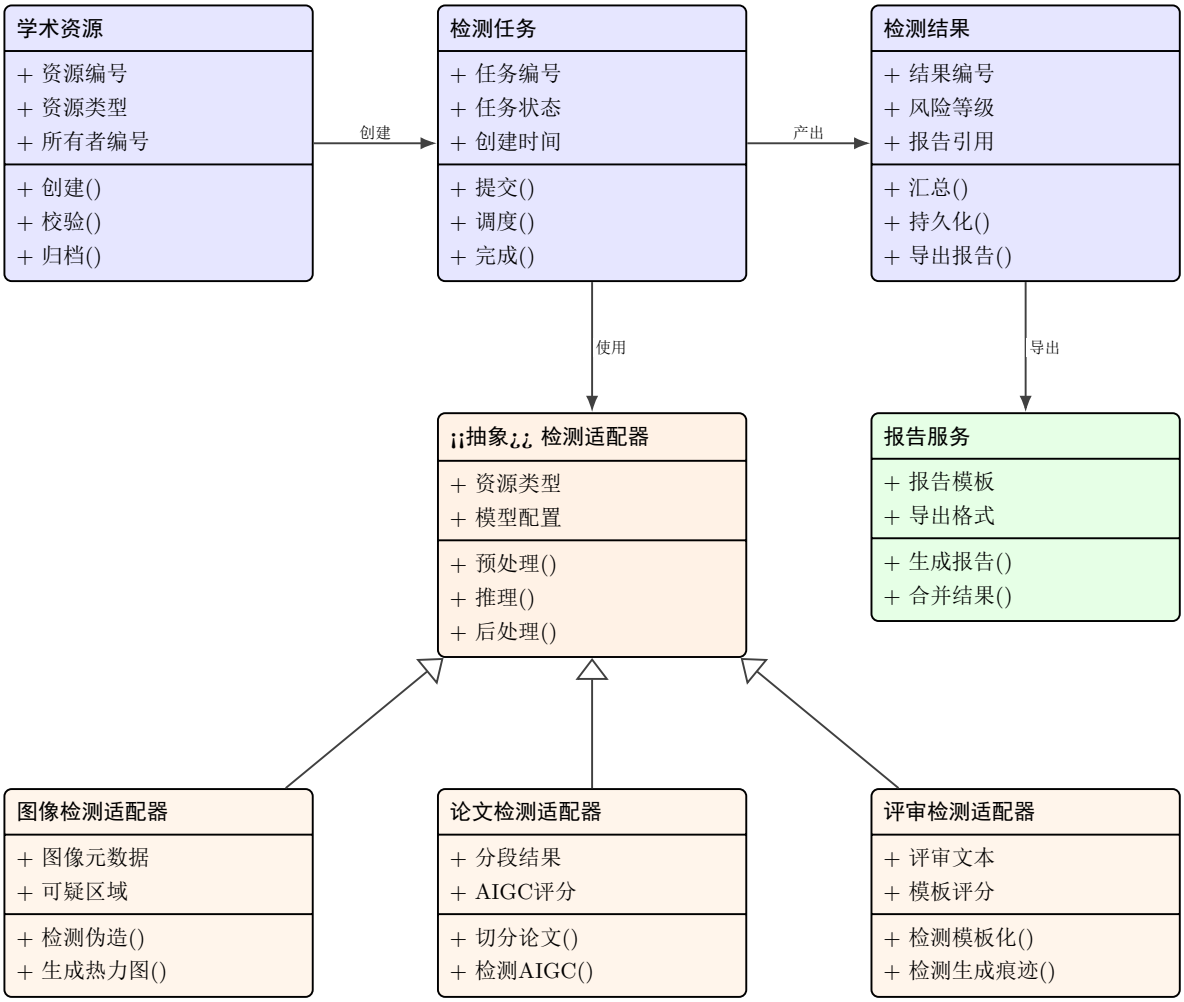

图 2: 统一资源与检测扩展关系 UML 类图

图 3-2 展示了本轮迭代在架构抽象层面最关键的一次调整，即将现有系统中偏图像中心的实现方式，提升为统一资源与统一检测任务的结构。具体而言，系统通过 **AcademicResource** 抽象统一承载图像、论文和 Review 三类对象，通过 **DetectionTask** 抽象统一管理检测请求和状态流转，通过 **DetectionResult** 抽象统一组织检测输出，并通过 **DetectionAdapter** 抽象隔离不同资源类型对应的模型调用差异。

这一设计的直接收益在于：后续新增一种检测对象时，不必改动整个任务链路和结果链路，只需要在统一任务框架下补充新的资源表示和适配器实现即可。这种方式比在现有图像任务逻辑上叠加条件分支更稳定，也更适合课程项目后续继续迭代。

3.5 系统组成

从逻辑结构上看，系统由以下几个核心组成部分构成：

- **用户端**：面向普通用户和审核参与者，负责资源上传、检测任务发起、结果查看、人工审核参与和通知展示；
- **管理端**：面向组织管理员和超级管理员，负责用户治理、组织管理、审核审批、日志查看、统

计分析和模型配置入口；

- **后端业务服务**：负责认证鉴权、资源管理、检测任务创建、结果聚合、报告生成、权限控制和管理接口暴露；
- **异步任务调度子系统**：负责长耗时任务的排队、拆分、执行、失败重试和状态同步；
- **AI 推理子系统**：负责图像、论文和 Review 等不同检测模型的具体执行；
- **消息与通知子系统**：负责检测状态推送、系统通知分发和用户消息反馈；
- **数据库与文件存储**：负责业务数据、日志、上传资源、检测中间产物和报告文件的持久化。

3.6 子系统职责划分

子系统	主要职责	典型技术实现
用户端子系统	面向普通用户和审核用户提供资源上传、检测发起、结果展示、审核参与和通知接收能力	Vue + Vuetify
管理端子系统	面向组织管理员和超级管理员提供治理、统计、日志查询和审核审批能力	Vue + Vuetify
业务服务子系统	提供统一 REST 接口，承担认证鉴权、权限控制、业务编排、结果封装和报告管理	Django + DRF
消息与实时通知子系统	负责 WebSocket 推送、任务状态广播、系统公告与用户通知分发	Django Channels
异步调度子系统	负责长时任务调度、批处理拆分、失败重试、执行收尾和状态回写	Celery + Redis
AI 推理子系统	负责检测模型推理、特征计算、风险分析和差异化结果生成	独立 Python 推理服务
数据存储子系统	负责业务数据、任务状态、日志记录、审核记录和元数据存储	MySQL
文件存储子系统	负责上传资源、结果文件、可视化产物和报告文件保存	本地媒体目录/文件系统

3.7 核心调用链设计

系统的核心调用链可概括为以下几个阶段：

1. 用户通过用户端或管理端发起业务请求，例如上传资源、提交检测、申请人工复核或审批审核任务；
2. 前端通过 HTTP API 或 WebSocket 将请求发送到后端业务服务；
3. 后端完成身份认证、权限校验、资源登记、任务创建和状态初始化，并根据任务性质决定同步处理还是异步处理；
4. 对于检测类长时请求，后端将任务提交到异步任务调度子系统，由调度子系统负责排队、分发和失败重试；

5. 异步任务调度子系统根据任务类型调用对应的检测适配器，由适配器进一步调用 AI 推理服务完成模型执行；
6. 推理结果经过后端业务服务统一汇总后写入数据库与文件存储，并触发报告生成、通知创建和状态变更；
7. 前端通过轮询接口或实时推送机制获取任务最新状态、检测结果和报告链接。

上述调用链将“用户请求响应速度”和“检测处理耗时”有效解耦，使平台既能保证交互上的及时反馈，又能处理较重的文件读写、模型推理和报告生成任务。

3.8 分层设计

为了使系统在结构上更清晰、在实现上更易维护，本系统采用分层设计。各层职责如下表所示：

层次	职责
表示层	提供用户端与管理端页面、表单交互、结果展示和状态反馈，仅负责界面组织与前端状态管理，不直接承载核心业务规则。
接口层	提供 HTTP API 与 WebSocket 接口，对外暴露统一访问入口，负责请求接收、参数校验和基础异常响应。
业务层	实现用户、组织、资源、检测、审核、报告、通知和统计等核心业务逻辑，是系统的业务中枢。
任务层	处理异步检测任务、批处理调度、状态同步、任务收尾和报告生成触发。
适配层	对不同资源类型对应的检测模型和外部推理能力进行适配与封装，屏蔽底层模型差异。
数据层	存储用户数据、任务数据、审核数据、日志、文件路径和报告元数据，保证业务数据可持久化和可追溯。

3.9 架构约束与关键设计决策

为了保证后续详细设计和编码实现能够沿同一方向演进，本系统在总体架构层面明确以下约束与设计决策：

- **前端不直接调用模型服务**。所有检测请求必须先进入后端业务服务，由后端统一完成权限校验、任务登记和状态管理；
- **检测必须通过统一任务中心流转**。无论检测对象是图像、论文还是 Review，原则上都应通过统一的任务创建、调度和回写机制执行；
- **结果主体与差异化载荷分离**。统一结果对象只保存通用状态、风险等级和报告引用，具体差异化内容由资源类型专属结构承载；
- **模型接入通过适配器隔离**。业务层不直接依赖具体模型实现，模型替换和模型扩展应限制在适配层及推理子系统内部；
- **人工审核链路独立于自动检测链路存在**。人工审核可以引用自动检测结果，但不应与模型执行过程耦合，从而保证人工复核流程可单独演进；

- **关键过程必须可追溯**。资源上传、任务创建、状态变化、审核结论和报告生成都需要在数据层保留可回查记录。

3.10 面向迭代的架构调整方向

考虑到当前实现仍以图像检测流程为中心，架构上需要逐步向统一资源架构过渡。该过渡主要体现在以下几个方面：

- **资源对象统一化**：从仅支持图像的资源表示过渡为统一支持图像、论文和 Review 的资源抽象；
- **检测能力适配化**：图像检测、论文检测和 Review 检测分别通过独立适配器接入，避免在业务代码中硬编码不同检测逻辑；
- **结果结构层次化**：采用统一结果主体与差异化结果载荷相结合的方案，使不同资源类型既能纳入统一任务流程，又能保留各自结果特点；
- **治理能力泛化**：管理端从围绕图像任务治理扩展为面向统一学术资源、统一日志和统一模型配置的治理结构；
- **报告生成标准化**：将现有偏图像结果的报告输出能力扩展为可面向多资源类型的统一报告服务。

3.11 关键架构优势

本轮总体架构设计具备以下优势：

- 能复用现有前后端、异步任务与图像检测链路，降低迭代开发成本；
- 能通过统一任务体系纳入论文和 Review 两类新资源，避免形成多个割裂子系统；
- 能将长时检测任务从用户请求链路中解耦，提高系统交互流畅性；
- 能将模型推理能力与业务编排逻辑分离，便于后续替换或新增模型；
- 能为后续详细设计中的数据库重构、接口扩展、实体建模和流程设计提供稳定基础。

3.12 总体架构小结

综上，本系统总体架构并不是对现有实现的简单罗列，而是对当前代码基础、迭代需求目标和后续演进方向进行综合平衡后的结果。其核心思想是：在保留当前图像检测平台工程基础的同时，通过统一资源、统一任务、统一结果和适配化模型接入，将系统演进为可持续扩展的综合学术 AI 鉴伪平台。

4 模块设计

4.1 模块划分原则

本系统的模块划分不按代码目录机械展开，而是按业务职责、数据边界和协作关系进行组织。其核心目标是将当前代码中已经存在的用户、组织、资源、检测、审核、通知和日志能力沉淀为稳定模块，同时为论文检测、评审文本检测和统一报告等迭代需求预留扩展空间。

在划分过程中遵循以下原则：

- **围绕业务能力划分模块**。模块对应稳定业务职责，而不是简单对应前端页面或后端文件夹；
- **高内聚、低耦合**。单个模块内部尽量完成同一类业务闭环，不直接依赖过多外部模块细节；
- **统一入口、差异化实现**。资源管理、检测任务管理和结果输出尽量统一，不同检测对象的差异通过专用检测模块或适配器处理；
- **兼顾现有实现与后续演进**。模块设计既要能映射当前 Django 后端、Vue 前端、Celery 任务和 AI 服务实现，也要能支撑下一阶段新增论文与评审能力。

4.2 模块总体关系类图

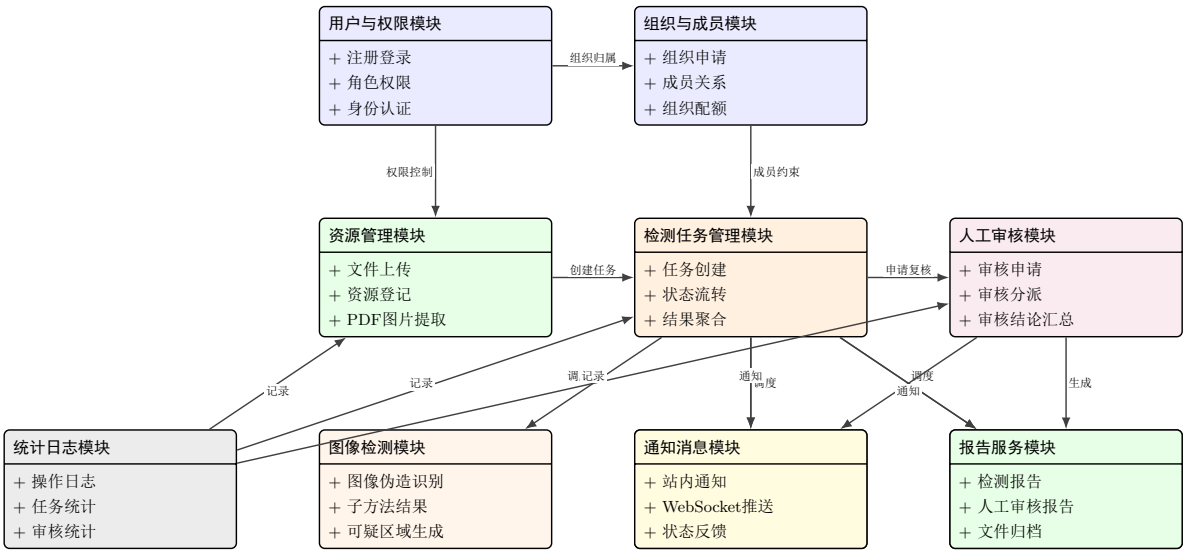

图 3: 系统模块关系 UML 类图

4.3 模块总览

本章模块标识符采用“SD-模块缩写-序号”的统一编码方式，其中“SD”表示概要设计阶段，模块缩写用于区分不同业务模块，序号用于标识该模块条目。

模块名称、标识符	主要职责	主要依赖
用户与权限模块 SD-YHQX-0001	管理账户全生命周期、身份认证与会话、角色及细粒度业务权限判定，为管理端与普通用户端提供统一安全访问控制。	组织与成员模块（用户组织归属与组织内策略上下文）
组织与成员模块 SD-ZZCY-0001	组织入驻申请与审批、组织档案与成员关系维护、组织级检测配额与组织内角色权限协同。	用户与权限模块（账户主体、管理员审批操作身份）

资源管理模块 SD-ZYGL-0001	对学术资源进行统一管理、存储、预处理，为后续提供一致的数据访问接口	用户与权限模块
检测任务管理模块 SD-JCRW-0001	负责鉴伪任务的创建、调度、状态跟踪等全生命周期管理	资源管理模块
图像检测模块 SD-TPJC-0001	对学术图像进行自动检测、子方法分析和可疑区域标注。	资源管理模块、检测任务管理模块、报告服务模块
论文检测模块 SD-LWJC-0001	对全篇论文进行 AIGC 检测、段落级风险分析和事实性鉴伪说明。	资源管理模块、检测任务管理模块、报告服务模块
评审检测模块 SD-PSJC-0001	对同行评审 Review 进行 AI 生成倾向和模板化倾向检测。	资源管理模块、检测任务管理模块、报告服务模块
人工审核模块 SD-RGSH-0001	待补充	待补充
报告服务模块 SD-BGFW-0001	待补充	待补充
通知消息模块 SD-TZXX-0001	统一承载任务状态、审核进展、组织审批和系统消息通知，并提供站内消息与实时推送能力	用户与权限模块、检测任务管理模块、人工审核模块、组织与成员模块
统计日志模块 SD-TJRZ-0001	记录关键业务操作与审计轨迹，并为管理端提供统计分析、趋势汇总和日志检索能力	用户与权限模块、组织与成员模块、检测任务管理模块、人工审核模块、通知消息模块

4.4 用户与权限模块（SD-YHQX-0001）

4.4.1 模块目标

落实迭代需求中管理端“用户管理”目标：区分管理员与普通用户，按角色配置可执行操作，保证系统可管可控与安全性；为用户端上传、检测、报告与人工审核等能力提供统一身份与权限判定入口。

4.4.2 模块职责

- 维护平台用户账户（注册、资料、头像、邮箱验证与密码找回等），并与 Django 认证体系对接；
- 管理角色模型（如管理员、发布者、审核者等）及与业务动作绑定的细粒度权限（如上传、提交检测、发布、参与审核等），在接口层统一鉴权；
- 签发与校验访问令牌与会话，向其他模块暴露当前用户身份、角色与权限摘要，支撑操作审计与资源访问边界。

4.4.3 功能组成

- 用户注册、登录、登出与令牌刷新，登录时可选择业务角色以匹配前端与权限策略；
- 用户资料维护、头像上传、个人检测配额展示（与是否归属组织及组织策略相关）；
- 管理端用户列表与检索、用户创建与编辑、按权限位筛选、用户删除与管理员专用账户维护；
- 管理端为用户增减或调整业务权限、查看用户操作日志（与统计日志模块协同）；
- 密码重置流程（邮箱验证码发放、校验与密码更新）。

4.4.4 输入与输出

输入：

- 注册与登录凭证（用户名、邮箱、密码、角色选择等）及令牌刷新请求；
- 用户资料变更、头像文件、密码重置邮箱与验证码；
- 管理端操作用的管理员身份及目标用户标识、权限字段或筛选条件。

输出：

- 认证成功后的访问令牌、刷新令牌及基础用户信息（含角色、权限摘要、组织标识等）；
- 用户详情、分页用户列表、操作日志查询结果及权限变更结果提示；
- 鉴权失败或无权访问时的统一错误响应，供网关与各业务视图复用。

4.4.5 核心数据对象

- `User`：扩展自 `AbstractUser`，承载用户名、邮箱、角色、组织外键、头像与简介等，并通过整型 `permission` 字段编码上传、提交、发布、审核等能力位；
- 令牌与验证码相关状态：访问令牌与刷新令牌由认证中间层管理；密码重置验证码及过期时间可存于用户记录；
- 与发布者–审核者协作相关的关联关系对象（如多对多经中间表），支撑业务侧“谁可为谁审核”的约束，仍属用户域数据。

4.4.6 处理流程

1. 用户提交注册或登录请求，模块校验凭据与角色合法性，通过后签发令牌并返回用户上下文；
2. 业务请求到达时，从令牌解析用户身份，结合角色与 `permission` 判定是否允许访问目标接口或资源；
3. 管理端发起用户或权限变更时，校验操作者具备管理员或超级管理员资格，再更新 `User` 及关联字段；
4. 密码重置时生成限时验证码并发送邮件，校验通过后更新密码并清除验证码状态；
5. 登出或刷新流程使旧令牌失效或轮换刷新令牌，降低凭证泄露风险。

4.4.7 模块接口与交互关系

- 对前端用户端、管理端暴露 REST 风格用户与认证接口（注册、登录、登出、资料、令牌刷新、密码重置等）；
- 与组织与成员模块协作：读取用户所属 `Organization`，用于登录响应展示与后续组织级配额、策略校验；
- 为资源管理、检测任务、人工审核、报告下载等模块提供统一鉴权钩子或中间件，避免各模块重复解析令牌；
- 与统计日志模块协作：关键管理操作写入操作日志，便于审计与问题排查。

4.4.8 设计要点

- 密码与敏感字段必须加密存储，令牌具备有效期并优先采用短期访问令牌加长周期刷新令牌组合；
- 角色默认值与权限位变更规则应集中维护，避免业务代码散落硬编码导致管理端与普通用户行为不一致；
- 管理员能力与普通用户能力严格分离，超级管理员操作应单独约束并留痕；
- 权限判定除用户自身字段外，需与组织归属一致考虑（无组织用户与组织成员在配额等策略上可能不同），具体组织规则由组织与成员模块实现，本模块消费其上下文；
- 接口错误信息应区分认证失败与授权失败，便于前端提示与安全审计。

4.4.9 后续扩展方向

- 引入更细 RBAC（角色–权限–资源）或策略引擎，支持按组织、按资源类型动态授权；
- 支持单点登录、多因素认证与企业目录对接，满足高校或期刊规模化部署需求。

4.5 组织与成员模块（SD-ZZCY-0001）

4.5.1 模块目标

支撑平台以组织为单位的账户治理与资源隔离：完成组织入驻与审批、成员加入与角色约束，并在组织维度管理检测调用配额，与迭代目标中高并发、多用户协作及可运营性要求一致。

4.5.2 模块职责

- 受理组织入驻申请，支持管理员审批、拒绝与归档，并在通过后创建正式组织及初始管理员账户；
- 维护组织主数据（名称、联系邮箱、描述、标识物料等）及组织与成员之间的归属关系；
- 维护组织级 LLM 与非 LLM 检测剩余次数等配额信息，为检测任务调度提供扣减与查询依据，并支持管理员调整组织维度权限模板。

4.5.3 功能组成

- 组织申请提交（组织信息、管理员账号、证明材料与 Logo 等可选附件）与申请状态查询；
- 管理端待审列表、检索、批准与拒绝组织申请；管理员直接创建组织等运维能力；
- 组织列表、详情、更新与删除，以及组织角色权限配置（与业务权限位协同）；
- 邀请码生成与消费、成员持码加入指定组织并绑定角色（如发布者、审核者）；
- 组织使用情况查询（剩余非 LLM 与 LLM 调用次数等），供用户端或管理端展示。

4.5.4 输入与输出

输入：

- 组织申请表单字段、证明文件与 Logo 上传流；管理员审批意见与操作类型（通过、拒绝）；
- 组织维护表单、组织级权限配置值、配额调整参数；
- 邀请码、用户加入组织时的身份与角色选择。

输出：

- 申请单状态、组织标识、组织公开信息及管理员关联结果；
- 组织成员列表或用户侧可见的组织名称、组织 ID 及配额摘要；
- 配额不足或无权操作组织资源时的业务错误信息，供检测任务等模块提前拦截。

4.5.5 核心数据对象

- `OrganizationApplication`：记录待审申请、提交时间、状态及处理人，关联证明材料与 Logo 等附件字段；
- `Organization`：正式组织实体，包含名称、邮箱、管理员用户一对一关联、描述、Logo、证明存档及组织级剩余调用次数等；
- `InvitationCode`：与组织、角色绑定的一次性邀请码及有效期、使用标记，用于受控扩员。

4.5.6 处理流程

1. 申请方提交 `OrganizationApplication`，系统持久化并通知管理端进入待审队列；
2. 管理员审核：拒绝则更新申请状态；批准则创建 `Organization`、建立管理员 `User` 与组织的关联，并视需要初始化配额；
3. 组织管理员或系统生成 `InvitationCode`，新用户凭码注册或绑定时写入用户的组织外键与角色；
4. 检测任务执行前后，由业务层调用组织配额校验与扣减（可与用户个人配额策略组合，以当前实现为准）；
5. 组织信息变更或删除时，级联或限制处理成员与历史资源引用，避免产生孤儿数据。

4.5.7 模块接口与交互关系

- 对外提供组织申请、审批、组织 CRUD、邀请码与组织使用情况等 REST 接口，供用户端与管理端调用；
- 依赖用户与权限模块：审批人、组织管理员及成员均以 `User` 为主体，认证鉴权由用户模块完成；
- 与检测任务管理模块、资源管理模块协作：在创建任务或上传资源时校验组织成员身份与组织配额；
- 与通知消息模块协作：申请结果、邀请相关事件可向用户推送站内通知或邮件（若已接入）。

4.5.8 设计要点

- 组织名称、联系邮箱等应保证唯一性或明确冲突处理策略，避免审批通过后无法落库；
- 申请中暂存的管理员密码等敏感信息需加密存储，审批拒绝后应及时清理或脱敏；
- 配额扣减与回滚应与任务状态机一致，防止任务失败仍扣次数或重复扣减；
- 删除组织前须评估成员、任务与文件的引用关系，采用软删除或迁移策略以保证学术资源可追溯；
- 组织级权限与用户级权限并存时，应文档化优先级（例如检测接口以组织配额为准还是以用户为准），减少前后端理解偏差。

4.5.9 后续扩展方向

- 支持多级组织、子部门与数据隔离域，满足大学院系或出版集团结构；
- 配额模型扩展为计费套餐、周期账单与弹性扩容，并与统计报表打通。

4.6 资源管理模块（SD-ZYGL-0001）

4.6.1 模块目标

本模块旨在建立学术资源的统一管理机制，实现对图像、论文（PDF）及评审文本（Review）的标准化存储、索引与预处理，为系统各鉴伪功能提供一致的数据访问接口。

4.6.2 模块职责

- 统一接入：处理各类学术资源的上传、校验与格式识别；
- 物理存储：负责文件在存储介质（本地或云端）的持久化；
- 元数据维护：记录资源指纹（MD5）、类型、归属人及引用关系；
- 内容解析：执行 PDF 图像提取、文本转码等特定预处理操作。

4.6.3 功能组成

- 资源上传与校验：支持分片上传，验证文件完整性及安全性；
- 自动化解析引擎：针对 Paper 类型资源，自动剥离 PDF 内嵌图像；
- 资源生命周期管理：提供资源的逻辑删除、过期清理及检索功能。

4.6.4 输入与输出

输入：

- 原始文件：用户上传的学术资源文件（如图像、文本等）；
- 资源类别；
- 所有者。

输出：

- 唯一标识ID（ResourceID）：为每个资源分配的唯一标识符，用于后续检索和管理。

4.6.5 核心数据对象

- `AcademicResource`（实体）：包含资源 ID、类型编码、存储路径、文件哈希值、父级资源 ID（针对提取出的子图）；
- `StorageConfig`（配置）：定义不同类型资源的存储策略（如存储位置、备份频率）。

4.6.6 处理流程

1. 模块接收到资源请求后，首先通过指纹校验判断是否为重复资源；
2. 若为新资源，则进行物理写入并同步至数据库；
3. 针对PDF论文，系统会异步触发解析任务，将其中的图片提取并重新注册为关联资源，最后返回处理结果。

4.6.7 模块接口与交互关系

- 与用户与权限模块的交互：在资源上传、下载或删除时，本模块需调用用户与权限模块验证当前操作者的身份（如Token/UID），确保资源的私密性与合规性；
- 与检测任务管理模块的交互：这是最核心的交互。资源管理模块为检测任务管理模块提供唯一标识ResourceID的查询接口。当鉴伪任务启动时，检测任务管理模块通过此接口获取文件的物理路径或内存流，从而将数据传递给算法检测层；
- 与图像、论文以及评审检测模块的交互：本模块为算法检测模块提供预处理后的标准化输入。例如，将PDF论文中提取出的图片序列化后，传递给图像检测模块。

4.6.8 设计要点

- 高性能存储：采用哈希目录结构存储文件，避免单目录下文件过多导致的性能下降；
- 解耦设计：资源处理逻辑与具体的AI算法解耦，确保资源模块的通用性。

4.6.9 后续扩展方向

- 资源指纹去重：引入感知哈希（pHash）技术，识别视觉相似的重复图片；
- 多存储后端支持：未来可以考虑计划接入阿里云或腾讯云提高可用性。

4.7 检测任务管理模块（SD-JCRW-0001）

4.7.1 模块目标

实现鉴伪任务的全生命周期管理，负责任务的创建、调度、状态追踪及结果汇总，确保 AI 算法能高效、有序地处理用户提交的学术资源。

4.7.2 模块职责

- 任务编排：根据用户选择的检测类型（图像查重/论文查重/评审分析），匹配对应的检测模块；
- 异步调度：管理检测队列，支持高并发下的任务排队与分发；
- 状态监控：实时记录任务进度（等待中、检测中、已完成、失败）。

4.7.3 功能组成

- 任务创建中心：接收 ResourceID 和检测参数，校验任务合法性并入库；
- 调度引擎：基于消息队列（如 Redis/RabbitMQ）将任务分发至空闲的算法节点；
- 通过WebSocket或轮询接口向前端同步实时检测进度。

4.7.4 输入与输出

输入：

- 资源标识（ResourceID）；
- 检测模式；
- 优先级。

输出：

- 任务编号（TaskID）；
- 任务状态（如“等待中”、“检测中”、“已完成”、“失败”）；。

4.7.5 核心数据对象

- `DetectionTask`（实体）：包含TaskID、关联ResourceID、当前状态、开始/结束时间、耗时、异常信息；
- `AlgorithmRegistry`（配置）：记录系统中可用的算法插件列表及其参数约束。

4.7.6 处理流程

1. 用户在界面发起检测请求后，该模块首先调用资源管理模块确认文件就绪；
2. 随后在数据库创建初始化状态的任务记录，任务进入待处理队列，由调度引擎根据算法负载情况取出并执行；
3. 算法执行完成后，模块更新任务状态为“已完成”，并触发报告服务模块的预处理。

4.7.7 模块接口与交互关系

- 与资源管理模块交互：通过ResourceID获取物理文件路径，确保检测对象存在；
- 与图像、论文以及评审检测模块交互：作为算法的“雇主”，通过接口调用具体的推理服务并接收回调；
- 与报告服务模块交互：任务完成后，将TaskID传递给报告服务模块，由其生成可视化报告。

4.7.8 设计要点

- 容错处理：若算法层超时或崩溃，模块需具备自动重试（Retry）机制或超时熔断逻辑；
- 解耦设计：采用“任务-算法”解耦架构，新增算法插件时无需修改任务管理的核心逻辑。

4.7.9 后续扩展方向

- 分布式调度：支持跨服务器的算法节点调度，应对大规模学术会议的检测峰值；
- 优先级体系：引入付费用户或紧急任务的优先处理通道。

4.8 图像检测模块（SD-TPJC-0001）

4.8.1 模块目标

本模块面向用户上传或系统解析得到的学术图像，完成自动化 AI 鉴伪、异常痕迹识别和可疑区域标注。该模块需要在现有图像检测能力基础上，为检测任务、结果展示、综合报告和人工审核提供结构化图像检测结果。

4.8.2 模块职责

- 接收资源管理模块登记的图像文件或从论文、压缩包、PDF 中提取出的图像资源；
- 调用图像检测模型或图像鉴伪算法，判断图像是否存在 AI 生成、复制伪造、拼接篡改、背景掩盖等风险；
- 记录图像检测的整体结论、置信度、子方法结果、可疑区域掩膜和可视化标注结果；
- 将图像检测结果回写到检测任务，并向报告服务模块、人工审核模块和通知消息模块提供结果数据。

4.8.3 功能组成

- 图像来源解析：对直接上传图像和从 PDF、压缩包、论文文档中抽取出的图像进行统一登记；
- 学术图像 AI 鉴伪：对图像进行整体风险判断，输出是否可疑、置信度和风险等级；
- 子方法检测结果汇总：保存多种检测方法的概率、掩膜矩阵和可视化结果；
- 图像可疑区域标注：在原图或标注图中展示可疑区域、区域编号、风险类型和局部说明。

4.8.4 输入与输出

输入：

- 图像资源文件及其元数据，包括文件名称、文件类型、上传用户、所属组织、来源页码或来源文件；
- 检测任务信息，包括任务编号、任务状态、检测参数、是否启用大语言模型辅助判断等配置；
- 图像检测服务所需的模型参数、图像预处理参数和可视化标注参数。

输出：

- 图像整体检测结果，包括是否可疑、置信度、检测时间、任务状态和文字判断说明；
- 子检测结果，包括各检测方法概率、掩膜图、掩膜矩阵、ELA 图像、LLM 可视化图像等解释性数据；
- 面向用户端、管理端和报告服务的结构化结果数据。

4.8.5 核心数据对象

- `FileManagement`：记录用户上传文件的名称、大小、类型、标签和组织归属；
- `ImageUpload`：记录图像文件、来源文件、来源页码、检测状态、审核状态和图像级结论；
- `DetectionTask`：记录一次检测任务的发起用户、任务状态、检测参数和报告文件；
- `DetectionResult`：记录图像整体检测结论、置信度、检测状态、可视化结果和人工审核关联；
- `SubDetectionResult`：记录不同子检测方法的概率、掩膜图片和掩膜矩阵。

4.8.6 处理流程

1. 用户上传图像或包含图像的文件后，资源管理模块完成文件校验、存储和图像资源登记；
2. 检测任务管理模块创建图像检测任务，并将任务状态置为待处理或进行中；
3. 图像检测模块读取图像资源，执行尺寸归一化、色彩空间转换、格式转换等预处理；
4. 系统调用图像检测模型或外部 AI 服务，生成整体风险判断、子方法概率和可疑区域信息；
5. 检测结果写入 `DetectionResult` 与 `SubDetectionResult`，同步更新图像和任务状态；
6. 报告服务模块读取图像检测结果生成报告，通知消息模块向用户推送任务完成状态。

4.8.7 模块接口与交互关系

- 与资源管理模块交互，获取已上传图像、PDF 提取图像或压缩包解析图像；
- 与检测任务管理模块交互，接收任务调度请求并回写任务状态；
- 与 AI 推理服务交互，提交图像文件和检测参数，接收检测概率、掩膜和可视化结果；
- 与报告服务模块交互，提供图像检测结论、可疑区域和解释性数据；
- 与人工审核模块交互，为用户提交人工审核申请和审核人员复核提供图像检测依据。

4.8.8 设计要点

- 检测过程应采用异步任务方式执行，避免大图、批量图像和外部模型调用阻塞用户请求；
- 图像资源、检测任务、检测结果和报告之间应保持可追溯关系，便于后续复核和问题排查；
- 子检测方法结果应以可扩展结构保存，支持后续新增检测方法或替换模型；
- 可疑区域标注应与原始图像保持坐标一致，避免前端展示、报告生成和人工审核之间出现位置错位；
- 模型调用失败、图像损坏、格式不支持等异常应回写明确状态，避免任务长期停留在进行中。

4.8.9 后续扩展方向

- 扩展图像检测模型适配器，使系统支持不同鉴伪模型、不同检测策略和模型版本切换；
- 将图像检测结果纳入统一资源与综合鉴伪结果模型，便于与论文检测和评审检测结果合并生成综合报告；
- 增强图像局部解释能力，支持像素级高亮、区域级风险说明和人工复核标注回写。

4.9 论文检测模块（SD-LWJC-0001）

4.9.1 模块目标

本模块面向用户上传的全篇论文，完成论文文本解析、AIGC 风险检测、段落级概率分析和事实性鉴伪说明。该模块是本轮迭代在现有图像检测基础上扩展出的文本检测能力，设计上应接入统一检测任务和报告生成链路。

4.9.2 模块职责

- 对论文文件或论文文本进行格式校验、文本抽取、结构化解析和段落切分；
- 调用论文 AIGC 检测模型，输出全文级 AI 贡献占比、整体风险结论和结果摘要；
- 生成段落级 AI 生成概率、疑似 AI 生成段落列表、事实性鉴伪子结论及原因说明；
- 将论文检测结果写入统一检测任务，并为结果展示、综合报告和人工审核提供数据。

4.9.3 功能组成

- 全篇论文 AIGC 检测：输出全文级检测结论、AI 贡献占比或等价风险指标；
- AI 生成段落识别：标识疑似 AI 生成段落及其段落编号、文本内容和原文位置；
- 段落级 AI 生成概率分析：为段落生成概率、风险分值或风险等级；
- 事实性鉴伪子结论生成：针对论文中的可疑内容形成条目化子结论；
- 事实性鉴伪原因说明：为子结论提供原因、依据摘要和关联段落说明。

4.9.4 输入与输出

输入：

- 用户上传的论文文件或论文文本，包含 PDF、DOCX、TXT 等系统支持格式；
- 论文资源元数据，包括标题、文件名、上传用户、所属组织、学科标签和任务编号；
- 论文检测参数，包括是否启用事实性鉴伪、段落切分策略、模型版本和阈值配置。

输出：

- 论文全文级检测结果，包括整体风险等级、AI 贡献占比、结果摘要和检测状态；
- 段落级检测结果，包括段落编号、段落内容、AI 生成概率、风险标签和原文定位信息；
- 事实性鉴伪结果，包括子结论、原因说明、证据摘要和关联段落；
- 可被综合报告、用户端详情页和人工审核模块复用的结构化结果数据。

4.9.5 核心数据对象

- `FileManagement`：复用现有文件管理对象记录论文文件的上传信息和归属关系；
- `DetectionTask`：复用统一检测任务对象记录论文检测任务的状态、发起人和报告文件；
- 论文资源对象：建议在后续数据设计中新增或扩展，用于记录论文标题、正文文本、解析状态和结构化段落；
- 论文检测结果对象：建议用于记录全文级风险、AI 贡献占比、检测摘要和模型版本；
- 段落检测结果对象：建议用于记录段落编号、段落内容、段落级概率、风险标签、事实性子结论和原因说明。

4.9.6 处理流程

1. 用户上传论文文件或提交论文文本后，资源管理模块完成格式校验、文件存储和资源登记；
2. 检测任务管理模块创建论文检测任务，并记录任务类型、检测参数和初始状态；
3. 论文检测模块解析论文文本，完成正文抽取、章节识别、段落切分和清洗；
4. 系统调用论文 AIGC 检测模型，生成全文级风险结论和段落级概率结果；
5. 系统基于高风险段落生成事实性鉴伪子结论和原因说明，并建立结果与原文段落的映射关系；
6. 检测结果写入统一任务结果，并触发报告生成、结果展示和通知推送。

4.9.7 模块接口与交互关系

- 与资源管理模块交互，接收论文文件、解析后的文本内容和论文元数据；
- 与检测任务管理模块交互，接收任务调度请求并更新论文检测状态；
- 与 AI 推理服务或文本检测适配器交互，提交论文文本、段落列表和检测参数；
- 与报告服务模块交互，提供全文结论、段落级结果、事实性子结论和原因说明；
- 与人工审核模块交互，为论文人工复核提供疑似段落、概率结果和事实性鉴伪依据。

4.9.8 设计要点

- 论文解析与模型检测应分阶段处理，解析失败、检测失败和结果生成失败应分别记录状态；
- 段落编号、段落内容、概率结果和事实性说明之间应保持稳定映射，避免前端高亮和报告内容错位；
- 论文文本可能包含公式、表格、参考文献和附录，检测前应明确正文抽取和噪声过滤策略；
- 论文内容属于敏感学术资源，结果展示、下载和人工审核流转应遵守用户权限和组织边界；
- 论文检测模型应通过适配器接入，避免后续替换模型时影响上传、任务和报告主流程。

4.9.9 后续扩展方向

- 增加论文专用数据表或统一资源子类型，用于保存论文结构、段落索引和检测版本；
- 支持多模型融合检测，将 AIGC 检测、事实性鉴伪、引用异常检测等能力统一聚合；
- 支持论文检测结果与图像检测结果联动，形成面向整篇论文的综合鉴伪报告。

4.10 评审检测模块（SD-PSJC-0001）

4.10.1 模块目标

本模块面向同行评审 Review 文本，检测其是否存在 AI 生成、异常生成或模板化倾向。该模块用于补充论文与图像检测之外的评审文本风险识别能力，为虚假评审识别、综合鉴伪报告和人工审核提供依据。

4.10.2 模块职责

- 接收用户上传或输入的同行评审文本，并完成格式校验、文本抽取和预处理；
- 调用评审文本检测模型，判断 Review 是否存在 AI 生成倾向、异常文本特征或模板化表达；
- 输出风险等级、置信度、可疑语句、模板化短语和结构化说明；
- 将评审检测结果纳入统一检测任务，并支持后续报告生成、日志追踪和人工复核。

4.10.3 功能组成

- 同行评审 Review 检测：识别评审文本是否存在 AI 生成或异常生成倾向；
- 模板化评审倾向识别：识别固定句式、高重复表达、常见模板短语和异常长度特征；
- 可疑语句定位：返回可疑句子、段落位置、风险标签和解释说明；
- 评审检测结果归档：保存检测状态、风险评分、模型输出和报告所需结构化数据。

4.10.4 输入与输出

输入：

- 用户上传的 Review 文本文件或在线输入内容，包含 TXT、DOCX、PDF 等系统支持格式；
- Review 元数据，包括上传用户、所属组织、任务编号、语言类型和可选的论文关联信息；
- 评审检测参数，包括 AI 生成阈值、模板短语库、相似度比对策略和模型版本。

输出：

- Review 整体检测结果，包括是否疑似 AI 生成、风险等级、置信度和结果摘要；
- 模板化倾向结果，包括模板化风险评分、高重复语句、模板短语匹配和相似文本提示；
- 可疑文本定位结果，包括句子或段落位置、风险标签和解释说明；
- 面向综合报告、用户详情页、管理端日志和人工审核的结构化结果数据。

4.10.5 核心数据对象

- `FileManagement`：复用现有文件管理对象记录 Review 文件的上传信息、类型和归属关系；
- `DetectionTask`：复用统一检测任务对象记录 Review 检测任务状态、用户和报告文件；
- Review 资源对象：建议在后续数据设计中新增或扩展，用于保存 Review 文本、语言类型、来源信息和解析状态；
- Review 检测结果对象：建议用于记录 AI 生成风险、模板化倾向、可疑语句、置信度和模型版本；
- 检测日志对象：用于记录评审检测过程、异常情况和管理端查询所需的追踪信息。

4.10.6 处理流程

1. 用户上传 Review 文件或输入 Review 文本后，资源管理模块完成格式校验和资源登记；
2. 检测任务管理模块创建 Review 检测任务，并记录任务类型、检测参数和初始状态；
3. 评审检测模块对文本进行清洗、分句、分词、语言标准化和结构化预处理；
4. 系统调用评审检测模型，计算 AI 生成概率、语义重复率、语言流畅度异常和模板化风险；
5. 系统生成整体风险结论、模板化倾向说明和可疑语句定位结果；
6. 检测结果写入统一任务结果，并触发报告生成、用户通知和日志记录。

4.10.7 模块接口与交互关系

- 与资源管理模块交互，获取 Review 文件、文本内容和元数据；
- 与检测任务管理模块交互，接收任务调度请求并回写检测状态；
- 与 AI 推理服务或文本检测适配器交互，提交 Review 文本和模板化检测参数；
- 与报告服务模块交互，提供整体风险、模板化结果、可疑语句和解释说明；
- 与统计日志模块交互，记录 Review 检测任务执行过程、异常信息和管理端检索数据。

4.10.8 设计要点

- Review 文本通常篇幅较短，检测设计应避免仅凭长度或固定阈值给出过度结论；
- 模板化检测应区分正常评审常用表达和异常批量生成表达，降低误判风险；
- 检测结果应提供可疑语句或模板化片段，不能只返回单一风险分数；
- 模板短语库、相似度比对策略和模型阈值应可配置，便于后续在管理端进行调整；
- Review 检测结果应与权限、组织边界和日志追踪机制保持一致，防止评审内容泄露。

4.10.9 后续扩展方向

- 支持中英文及多语言 Review 检测，并根据语言类型选择不同模型或模板规则；
- 支持历史 Review 相似度比对，用于发现批量复制、模板化改写和虚假评审行为；
- 将 Review 检测结果与论文检测、人工审核意见合并，形成面向投稿或评审流程的综合风险判断。

4.11 人工审核模块（SD-RGSH-0001）

4.11.1 模块目标

本模块的目标是实现学术资源的人工审核功能，支持图像、论文和评审结果的人工审核。通过专家审核任务分配与管理，确保AI检测结果的准确性与可信度。

4.11.2 模块职责

- **人工审核任务管理：** 分配和管理用户提交的人工审核请求，包括图像、论文和评审的审核。
- **专家审核任务分配：** 根据AI检测结果分配审核任务，确保任务按时完成并汇总结果。
- **审核结果汇总：** 汇总所有专家的审核结果，生成最终的人工审核报告。

4.11.3 功能组成

- **功能点 1：任务分配：** 根据AI检测结果分配人工审核任务给相关领域的专家，保证任务的针对性与效率。
- **功能点 2：专家审核：** 专家依据AI检测结果进行人工审核，标注图像问题、分析论文内容或评审质量，最终提交审核报告。
- **功能点 3：审核报告生成：** 汇总专家的审核结果并生成详细的人工审核报告，提供最终结论。

4.11.4 输入与输出

输入：

- AI检测结果（图像/论文/评审）；
- 审核任务分配信息（专家任务分配、任务详情）。

输出：

- 审核结果（人工审核报告）；
- 审核任务完成情况（专家评分、审核意见）。

4.11.5 核心数据对象

- **审核任务：** 记录审核任务的基本信息，包括任务类型、分配专家、审核状态等。
- **专家审核结果：** 包含专家的评分、审核意见、标注信息等。
- **人工审核报告：** 汇总所有专家审核意见的最终报告，包含检测结论、建议等。

4.11.6 处理流程

1. 系统根据AI检测结果生成人工审核任务，并分配给相关专家；
2. 专家接收任务并进行人工审核，标注问题并提供审核意见；
3. 专家完成审核后提交结果，系统汇总所有专家的审核报告；
4. 最终审核结果生成报告，供用户查看与下载。

4.11.7 模块接口与交互关系

- **任务分配接口：** 系统与任务分配平台交互，分配审核任务；
- **审核提交接口：** 专家通过系统提交审核结果，上传标注内容；
- **报告生成接口：** 将专家审核结果生成最终报告，并提供下载接口。

4.11.8 设计要点

- 确保任务分配的精确性和及时性，保证审核任务高效流转；
- 审核结果的可追溯性，确保每个专家的评分和评论能够追溯和验证；
- 设计合理的专家任务管理系统，支持审核进度跟踪。

4.11.9 后续扩展方向

- 增强专家审核质量控制，提供更多维度的专家评价；
- 支持多语言或跨领域的专家审核。

4.12 报告服务模块（SD-BGFW-0001）

4.12.1 模块目标

报告服务模块的目标是为用户提供检测报告的生成、存储和下载功能。报告涵盖AI生成内容检测、图像篡改识别、评审验证等方面，帮助用户深入理解学术资源的真实性。

4.12.2 模块职责

- **报告生成：** 根据AI检测结果和人工审核结果生成综合报告；
- **报告存储：** 存储生成的报告，供用户随时访问；
- **报告下载：** 提供下载接口，允许用户下载PDF或其他格式的报告。

4.12.3 功能组成

- **功能点 1：生成综合报告：** 汇总AI检测结果与人工审核结果，生成详细的报告。
- **功能点 2：报告格式支持：** 支持多种格式的报告输出，如PDF、Word等，满足用户需求。
- **功能点 3：报告管理：** 提供报告存储和查询功能，用户可以查看历史报告，管理报告。

4.12.4 输入与输出

输入：

- AI检测结果；
- 人工审核结果；
- 用户请求（如报告下载请求）。

输出：

- 综合报告（PDF/Word格式）；
- 报告查询结果。

4.12.5 核心数据对象

- **报告：** 包括报告的基本信息（报告ID、生成时间、相关检测任务等）。
- **检测结果：** AI检测和人工审核结果，用于生成报告的核心数据。
- **报告格式：** 定义不同格式报告的模板和输出方式。

4.12.6 处理流程

1. 用户提交检测任务，系统进行AI检测并生成初步报告；
2. 如果需要，人工审核任务将触发，专家审核结果将合并入报告；
3. 系统根据最终结果生成报告，并存储在报告管理系统中；
4. 用户可以随时查看和下载报告。

4.12.7 模块接口与交互关系

- **生成报告接口：** 与报告生成引擎交互，生成最终报告；
- **报告存储接口：** 将生成的报告存储到数据库中，并提供查询接口；
- **报告下载接口：** 提供下载接口，用户可以下载生成的报告。

4.12.8 设计要点

- 确保报告内容的完整性与准确性，避免任何遗漏；
- 提供多种报告格式支持，满足不同用户的需求；
- 加强报告的安全性和权限控制，确保用户隐私。

4.12.9 后续扩展方向

- 增加更多格式的报告支持，如Excel、HTML等；
- 提供定期报告生成功能，帮助用户跟踪检测历史。

4.13 通知消息模块（SD-TZXX-0001）

4.13.1 模块目标

本模块用于承载系统内与检测任务、人工审核、组织审批和管理操作相关的消息通知能力，保证用户能够及时获知任务状态变化、审核进展和系统事件，从而支撑平台的可用性与协作效率。

4.13.2 模块职责

- 统一接收业务模块产生的通知事件，并将其转换为站内消息记录；
- 为用户端和管理端提供通知列表、未读数量、标记已读与消息跳转能力；
- 通过 WebSocket 与轮询接口向前端同步关键业务状态变化，降低用户等待过程中的信息不透明问题。

4.13.3 功能组成

- 消息生成：根据检测完成、人工审核申请、组织审批结果、管理员操作等事件生成通知；
- 消息消费：提供通知列表查询、未读数统计、消息详情查看和已读标记能力；
- 实时推送：通过 WebSocket 向在线用户主动推送状态更新，离线场景下由前端通过接口补拉消息。

4.13.4 输入与输出

输入：

- 来自检测任务、人工审核、组织审批、后台管理等模块的业务事件；
- 当前用户身份、目标接收者、消息类别、消息标题、消息正文和跳转链接等消息元数据。

输出：

- 持久化后的通知记录、未读状态统计结果和消息查询结果；
- 面向前端的实时推送事件或标准 REST 响应，供页面更新通知中心与状态提示。

4.13.5 核心数据对象

- `Notification`：系统通知实体，记录收发双方、消息类别、标题、正文、已读状态、发送时间与跳转地址；
- 业务事件上下文：由其他模块传入的任务编号、审核请求编号、组织申请编号等引用信息，用于拼装通知内容；
- WebSocket 连接上下文：用于识别在线用户、定位推送目标并维护消息到达的实时性。

4.13.6 处理流程

1. 业务模块在状态变化或关键操作完成后发出通知事件，并传入接收者、类别和业务上下文；
2. 本模块根据事件类型组装通知标题、正文与跳转链接，生成 `Notification` 记录；
3. 若接收者在线，则通过 WebSocket 向对应会话推送消息摘要；若接收者离线，则等待其下次登录后通过查询接口获取；
4. 用户端或管理端读取通知中心数据，并可将消息标记为已读或跳转至对应业务页面。

4.13.7 模块接口与交互关系

- 接收检测任务管理模块、人工审核模块、组织与成员模块和管理端治理接口发出的消息事件；
- 对前端提供通知列表、未读消息数量、消息状态更新和 WebSocket 推送接口；
- 与用户与权限模块协同，校验当前访问者身份，确保通知只能被对应接收者或管理员读取。

4.13.8 设计要点

- 通知内容应避免直接复制大段业务数据，正文只保留必要摘要，详细信息由跳转页面展示；
- 同一业务事件的重复触发需有幂等控制，避免因任务重试或状态重复回写产生通知风暴；
- 实时推送与持久化消息应解耦，推送失败不影响消息记录落库；
- 通知分类需要兼顾当前实现与后续扩展，至少支持系统消息、全局公告、发布者到审核者和审核者到发布者等类型。

4.13.9 后续扩展方向

- 扩展邮件、短信或企业微信等多渠道通知，以适配组织级部署场景；
- 引入通知模板中心与用户通知偏好配置，实现按事件类型的订阅与静默策略。

4.14 统计日志模块（SD-TJRZ-0001）

4.14.1 模块目标

本模块用于记录系统关键操作、检测过程和人工审核过程中的审计信息，并向管理端提供统计分析能力，为运行监控、问题追踪和后续治理决策提供数据支撑。

4.14.2 模块职责

- 统一记录上传、检测、人工审核、组织审批和后台治理等关键操作日志；
- 对任务数量、检测结果、审核分布、用户活跃度等业务指标进行汇总统计；
- 为管理员提供可检索、可筛选、可追溯的日志与统计数据访问入口。

4.14.3 功能组成

- 日志采集：采集用户上传、检测提交、审核处理、组织审批、管理操作等事件；
- 统计分析：对用户数、检测数、审核数、检测结论分布和时间趋势进行聚合计算；
- 管理查询：支持管理端按时间、用户、组织、操作类型和业务对象进行筛选查询。

4.14.4 输入与输出

输入：

- 各业务模块产生的操作事件、状态变更事件和业务主键引用；
- 管理端提交的统计维度、时间范围、筛选条件和排序参数。

输出：

- 持久化日志记录、审计轨迹和按条件检索后的日志列表；
- 面向管理端仪表盘的统计汇总结果，如趋势数据、分布数据和总量数据。

4.14.5 核心数据对象

- Log：操作日志实体，记录操作时间、用户、操作类型、关联模型名称和关联对象编号；
- 业务统计聚合结果：由检测任务、检测结果、人工审核、用户与组织等实体汇总得到的统计数据；
- 管理端查询条件对象：封装筛选范围、排序方式、分页参数和维度组合。

4.14.6 处理流程

1. 用户或管理员完成上传、检测、审核、组织审批等操作后，业务模块生成对应日志事件；
2. 本模块将事件映射为统一的 Log 记录并写入数据库，必要时补充关联对象信息；
3. 管理端根据时间、组织、用户或事件类型查询日志或调用统计接口；
4. 本模块执行聚合分析后返回仪表盘数据、列表数据或导出所需的数据集。

4.14.7 模块接口与交互关系

- 接收用户与权限、组织管理、资源管理、检测任务、人工审核和通知模块写入的操作事件；
- 向管理端提供日志查询接口、统计接口和数据概览接口；
- 与报告服务模块和后续运维模块协同，为阶段性汇总报告和系统运行分析提供原始数据。

4.14.8 设计要点

- 日志字段设计应突出“谁在何时对什么做了什么”，优先保证可追溯性而不是过度冗余；
- 高频统计查询应与原始日志写入解耦，避免管理端分析请求影响主业务接口性能；
- 统计口径需在文档中固定，如检测次数按任务计还是按图片计，以免前后端和报告口径不一致；
- 涉及个人信息和敏感业务数据时，日志展示需做脱敏或按角色限制访问范围。

4.14.9 后续扩展方向

- 增加按组织、学科标签、模型版本和检测类型的多维统计分析能力；
- 支持日志归档、离线分析和异常行为告警，为后续运维监控与审计合规提供基础。

4.15 本章小结

本章在总体架构基础上，将系统划分为用户与权限、组织与成员、资源管理、检测任务、图像检测、论文检测、评审检测、人工审核、报告服务、通知消息和统计日志等核心模块，并统一说明了各模块的职责边界、输入输出、核心对象和协作关系。通过该划分，当前项目中已经落地的图像检测能力可以继续稳定运行，而论文检测、评审检测、统一报告和治理统计等迭代需求也具备了清晰的落点。

后续详细设计阶段，可在本章模块边界基础上继续细化数据库表结构、接口参数、状态机和异常处理策略，并按模块逐步推进开发与联调，以降低后续迭代时的耦合风险。

5 数据设计

5.1 数据设计目标

数据设计需要同时满足以下目标：一是支撑当前已经实现的图像上传、图像检测、人工审核、通知与管理统计能力；二是为后续论文检测、评审文本检测和统一报告生成预留稳定的数据抽象；

三是保证异步任务处理过程中的状态可追踪、结果可回溯和日志可审计；四是在后续详细设计阶段能够平滑从当前图像中心化模型演进到更统一的学术资源模型。

5.2 数据设计原则

- **兼容现有实现。** 当前后端已经围绕图像检测链路建立了较完整的数据模型，第五章设计需要以现有模型为基础，而不是完全脱离代码重新定义；
- **抽象统一资源。** 在保留 `FileManagement`、`ImageUpload` 等现有实体的同时，逐步抽象出统一的学术资源概念，以便后续纳入论文与评审文本；
- **状态可追踪。** 对文件、任务、检测结果、审核请求和通知状态进行显式建模，保证任务流转、回滚和异常排查都有数据依据；
- **便于扩展。** 新增检测类型、模型版本、报告格式或组织级治理能力时，应尽量通过增加关联实体或扩展字段完成，而不是破坏已有主链路。

5.3 核心数据模型类图

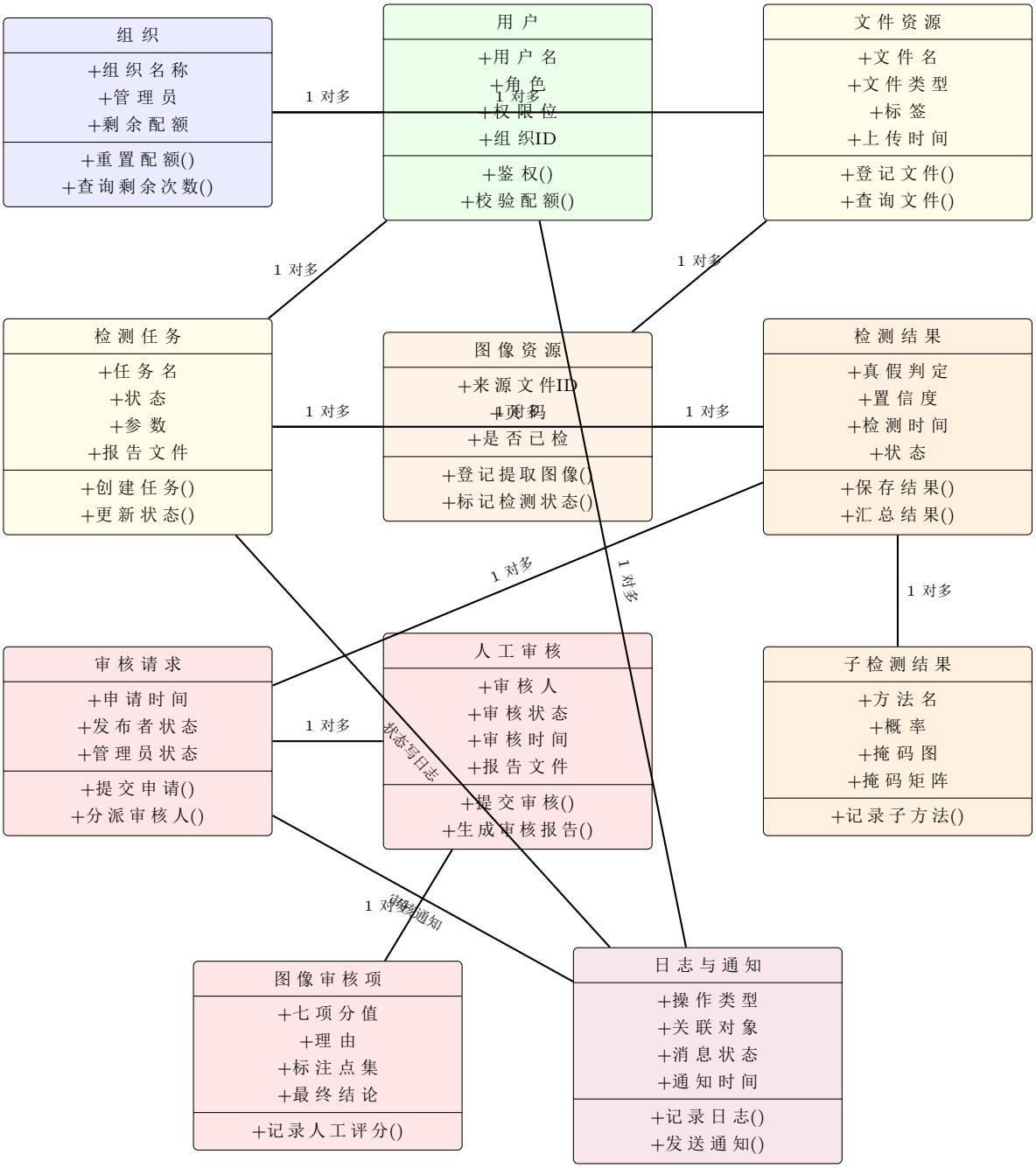

图 4: 核心数据模型 UML 类图

5.4 核心实体总览

实体名称	当前实现映射	说明
用户	`User`	系统基础身份实体，包含角色、权限位、组织归属、头像、简介、密码重置与个人配额信息。

组织与申请	OrganizationApplication、Organization、InvitationCode	支撑组织入驻审批、正式组织管理、成员邀请与组织级配额控制。
发布者-审核者关系	PublisherReviewerRelationship	用于限定发布者和审核者之间的协作关系，支持人工审核分派。
文件资源	FileManagement	当前项目中的一级资源登记实体，记录原始文件名、大小、类型、标签、上传者与组织。
检测任务	DetectionTask	记录一次检测提交的任务名称、状态、检测参数、报告文件和归属用户。
图像资源	ImageUpload	当前项目中的二级资源实体，既可表示用户上传的单张图像，也可表示从 PDF 中提取出的页面图像。
检测结果	DetectionResult、SubDetectionResult	存储图像级结论、置信度、ELA/LLM/EXIF 结果以及各子方法的细粒度输出。
审核流程	ReviewRequest、ManualReview、ImageReview、Feedback	承载人工审核申请、审核执行、逐图评分与审核反馈。
日志与通知	Log、Notification	支撑系统审计、管理统计和用户消息触达。

5.5 核心实体说明

5.5.1 用户与组织域

用户域数据以 User 为中心，记录账号身份、业务角色、权限位、所属组织和个人级剩余检测次数。组织域则由 OrganizationApplication、Organization 与 InvitationCode 组成，分别覆盖组织申请、正式组织信息与成员邀请机制。该设计使系统能够同时支持个人用户和组织用户两种使用模式，并为管理端组织审批和配额控制提供数据支撑。

5.5.2 资源与任务域

当前项目尚未落地统一的 Resource 实体，而是通过 FileManagement 与 ImageUpload 两层结构表达资源关系：前者代表用户上传的原始文件，后者代表参与检测的具体图像对象。对于 PDF 文件，系统会在上传后提取其中的图像页并为每张图建立 ImageUpload 记录。检测任务由 DetectionTask 表示，一个任务通常关联多个 ImageUpload 记录，并统一记录状态和参数。

5.5.3 结果与审核域

自动检测结果以 DetectionResult 为主实体，保存单张图像的真假判定、置信度、LLM 说明、可视化结果和 EXIF 判断。若需要展示多算法结果，则通过 SubDetectionResult 按方法逐条展开。人工审核链路以 ReviewRequest 发起，随后由一个或多个 ManualReview 承接执行，并通过

ImageReview 保存逐图评分、理由、标注点和人工结论，最终可由 Feedback 记录用户对审核结果的反馈。

5.5.4 治理与协同域

Log 用于审计谁在何时对哪个对象执行了何种操作，Notification 用于记录站内消息内容、收发双方、类别与未读状态。二者分别支撑管理端治理和用户侧消息协同，是后续统计分析、运行监控和问题追踪的重要基础。

5.6 实体关系说明

本系统的数据关系可以概括为“用户/组织 -¿ 资源 -¿ 任务 -¿ 结果 -¿ 审核 -¿ 报告/通知/日志”的链式结构。

- 一个组织可以拥有多个用户，一个用户最多归属于一个组织；
- 一个用户可以上传多个 FileManagement 记录，一个原始文件可以派生出多个 ImageUpload 记录；
- 一个 DetectionTask 由一个用户发起，可关联多个图像资源，并产生多个 DetectionResult；
- 一个 DetectionResult 可以继续关联多个 SubDetectionResult，用于表达多方法检测细节；
- 一个 ReviewRequest 针对一个检测结果发起，并可关联多个审核者和多个被审核图像；
- 一个 ManualReview 归属于一个审核请求与一个审核员，并进一步关联多个 ImageReview；
- 用户、任务、审核请求和管理操作均可派生出对应的日志记录与通知消息。

5.7 关键数据项设计

数据项	归属实体	设计说明
角色与权限位	User	通过 role 与 permission 组合描述用户在上传、提交、发布、审核等操作上的权限边界。
组织剩余配额	Organization	记录组织剩余 LLM 和非 LLM 检测次数，用于检测任务提交前的配额校验。
个人剩余配额	User	用于未加入组织用户或个人级额度控制场景，和组织配额形成双层约束。
文件标签与类型	FileManagement	当前通过 file_type 与 tag 管理资源类别，支撑分类、检索和学科统计。
检测参数	DetectionTask	通过 cmd_block_size、urn_k、if_use_llm 等字段记录任务配置。
真假判定与置信度	DetectionResult	直接表达自动检测主结论，是报告、列表页与后续审核分流的核心依据。
子方法概率与掩码矩阵	SubDetectionResult	支撑检测细节解释和可疑区域可视化，便于结果追溯。

审核双状态	ReviewRequest	通过发布者侧状态与管理员侧状态分离表达审核申请流转。
逐图评分与标注点	ImageReview	细化人工审核过程，使审核结论具备解释性和可复核性。
消息类别与已读状态	Notification	区分系统通知、全局通知与用户间通知，并支持通知中心未读管理。

5.8 状态设计

- **资源状态：** 当前代码中未单独定义文件资源状态，但从业务过程上可区分为“已上传、已解析、可检测、已归档”等逻辑状态，后续统一资源模型时应显式落表；
- **任务状态：** DetectionTask 当前定义为 pending、in_progress、completed，后续建议补充 failed、cancelled 等异常状态；
- **检测结果状态：** DetectionResult 当前区分 in_progress 和 completed，并通过 is_under_review 表示是否进入人工审核；
- **审核状态：** ReviewRequest 通过 status1 和 status2 分别表示发布者侧流程状态与管理员审批状态，ManualReview 进一步记录审核员执行状态；
- **通知状态：** Notification 采用 unread/read 两态，满足通知中心场景需要。

5.9 数据流转设计

从数据流角度看，当前项目的核心链路如下：

1. 用户登录后上传原始文件，系统生成 FileManagement 记录；
2. 若上传对象为 PDF，则系统提取其中图像并生成若干 ImageUpload 记录；若为单图，则直接登记图像记录；
3. 用户提交检测请求时创建 DetectionTask，并将待检测图像与任务建立关联；
4. 异步检测完成后，系统为每个图像生成 DetectionResult 与若干 SubDetectionResult；
5. 若用户申请人工审核，则创建 ReviewRequest，并进一步生成 ManualReview 和 ImageReview；
6. 在上述过程中，关键业务动作同步写入 Log，关键状态变化则生成 Notification，供前端展示和管理端统计。

5.10 完整性与一致性约束

- 用户邮箱、组织名称、组织邮箱和邀请码应保持唯一，避免身份与组织管理冲突；
- 检测结果必须关联到有效的图像资源，子检测结果必须依附于有效的检测结果；
- 审核请求、人工审核与图像审核之间应按流程顺序产生，禁止脱离上游对象单独创建下游数据；

- 配额扣减与任务状态更新需要保持一致，任务创建失败或执行失败时应避免发生重复扣减或脏数据；
- 通知与日志作为辅助数据，不应反向控制主业务，但其引用对象应能追溯到有效的主实体。

5.11 索引与查询设计

- 当前模型已经在组织外键、用户外键、申请时间、审核时间等高频过滤字段上使用了索引设计；
- 管理端高频查询通常围绕“按用户、按组织、按时间范围、按任务状态、按审核状态”展开，后续详细设计阶段可继续补充组合索引；
- 对于日志与通知类高增长表，应优先保证按时间倒序和按接收者查询的性能；
- 对于检测结果和子检测结果，应重点优化按任务编号、图像编号和审核请求编号的关联查询。

5.12 数据安全与归档设计

- 用户密码、重置验证码、组织证明材料等敏感信息需严格依赖后端认证机制和文件访问控制；
- 报告文件、审核材料、掩码图和上传图像属于业务关键数据，应保留稳定的文件路径和元数据索引；
- 对于已完成任务的历史结果、日志和通知，可采用归档或分区策略降低在线表压力；
- 日志展示和统计导出时应控制个人隐私字段暴露范围，满足课程项目场景下的基本合规要求。

5.13 兼容与演进说明

考虑到当前代码实现仍以图像资源为中心，后续详细设计阶段建议采用“兼容扩展”而非“推倒重建”的方式推进：

- 短期内继续复用 `FileManagement`、`ImageUpload`、`DetectionTask` 和 `DetectionResult`，保证现有图像检测闭环稳定；
- 中期可增加统一资源抽象实体，将论文文件、评审文本和图像资源纳入同一资源域，再通过资源类型字段区分不同检测对象；
- 人工审核、报告和通知模块的数据设计应尽量面向任务与资源抽象，而不是只绑定图像实体，以降低后续支持论文和评审检测时的迁移成本；
- 在模型升级过程中，应优先采用新增表、补充字段、增加映射关系等方式渐进演化，避免直接破坏既有接口和前端页面。

5.14 本章小结

本章结合当前后端模型，对用户、组织、资源、任务、结果、审核、日志与通知等核心数据对象及其关系进行了说明，并给出了状态设计、流转设计、一致性约束与演进方向。该数据设计既能够解释当前图像检测系统的实现基础，也为后续论文检测、评审检测和统一资源抽象提供了可继续细化的结构框架。

6 接口设计

6.1 接口设计目标

本系统接口设计的目标，是在当前项目已有的用户端、管理端、后端业务服务、异步任务调度和实时通知基础上，形成统一、清晰、可扩展的接口体系。接口设计既要服务当前已实现的图像上传、图像检测、人工审核、通知与管理功能，也要为论文检测、Review 检测、综合鉴伪报告和模型扩展预留统一接入方式。

从系统边界上看，本章接口设计主要覆盖三类接口：

- 面向前端用户端和管理端的 HTTP REST 接口；
- 面向前端的 WebSocket 实时推送接口；
- 面向系统内部的异步任务调度接口、AI 推理调用接口和文件访问接口。

6.2 接口设计原则

- **统一入口原则**：系统外部接口统一经由后端业务服务对外暴露，前端不直接访问数据库或 AI 推理服务；
- **REST 风格优先**：外部 HTTP 接口优先采用资源路径加标准方法的方式组织，便于前后端协作与后续接口治理；
- **同步与异步分离**：上传、查询、审批等短时操作采用同步接口；检测任务、报告生成等长时操作通过异步任务接口和状态查询接口解耦；
- **统一任务、差异结果**：接口层对不同检测对象尽量采用统一任务入口、统一状态查询接口，并通过差异化结果接口返回图像、论文和 Review 特有数据；
- **权限前置**：用户认证、角色检查、组织边界和资源归属校验应在接口层或接口进入业务层前完成；
- **可演进性**：接口命名和路径设计应兼顾当前实现与后续统一资源抽象，不将接口永久绑定到单一图像对象模型上。

6.3 接口总体分层

从调用关系看，本系统接口可划分为四层：

层次	说明
前端访问层	面向用户端与管理端提供 HTTP API 和 WebSocket 接口，负责请求接入、参数传递和结果展示。
业务接口层	由 Django + DRF 暴露具体接口，负责认证鉴权、参数校验、业务编排和响应封装。
系统协作层	负责后端与 Celery 任务、WebSocket 通道、文件系统和报告生成逻辑之间的协作。
能力适配层	负责后端与具体 AI 推理服务、图像检测算法、论文检测模型和 Review 检测模型之间的调用适配。

6.4 外部接口分类

当前项目外部接口可分为以下几类：

- 用户认证与个人信息接口；
- 组织与邀请码接口；
- 资源上传、查询与资源管理接口；
- 检测任务提交、状态查询、结果查询和报告下载接口；
- 人工审核申请、审核执行与审核查询接口；
- 通知与消息接口；
- 管理端用户管理、资源治理、仪表盘统计和日志管理接口。

当前后端主路由由 `fake_image_detector/urls.py` 将 `core.urls` 挂载在 `/api/` 下，因此所有 HTTP 接口统一以 `/api/` 作为访问前缀；实时通知接口通过 WebSocket 路由 `/ws/notifications/` 暴露。

6.5 内部接口分类

系统内部接口主要包括：

- **后端与异步任务系统接口**：由后端创建检测任务、投递 Celery 异步任务，并在任务完成后回写状态和生成通知；
- **后端与 AI 推理服务接口**：由图像检测模块、后续论文检测模块和 Review 检测模块调用对应推理服务；
- **后端与文件存储接口**：用于读取上传文件、提取图像、生成报告、保存中间产物和对外下载；
- **后端与消息通道接口**：用于向 WebSocket 消费者发送检测完成通知、审核任务通知和系统广播消息。

6.6 统一接口约定

为了保证接口风格一致，本系统采用如下约定：

- HTTP 接口统一返回 JSON 结构，文件下载接口返回文件流；
- 认证接口返回访问令牌、刷新令牌或用户上下文，业务接口通过认证信息识别当前用户；
- 涉及文件上传的接口使用 `multipart/form-data`；
- 涉及列表展示的接口支持分页或分页式响应结构；
- 涉及任务状态的接口应明确返回 `pending`、`in_progress`、`completed` 等状态字段；
- 错误响应应区分认证失败、权限不足、资源不存在、参数错误和系统异常。

6.7 当前项目外部接口总览

接口类别	访问方式	说明
用户认证接口	HTTP	提供注册、登录、登出、令牌刷新、用户资料查询与更新、头像更新和密码重置。
组织管理接口	HTTP	提供组织申请、审批、组织详情、邀请码和组织权限更新。
资源上传接口	HTTP	提供文件上传、文件详情查询、图片提取结果查询、文件标签更新和文件删除。
检测任务接口	HTTP	提供图像检测提交、任务状态查询、任务结果列表、任务删除和报告下载。
人工审核接口	HTTP	提供审核申请创建、审核任务列表、审核详情、审核提交、人工审核报告生成等能力。
通知接口	HTTP + WebSocket	提供通知列表、未读数量、已读状态更新、广播通知和实时推送。
管理端治理接口	HTTP	提供管理员登录、用户管理、文件管理、日志查询、仪表盘统计和审核审批。

6.8 用户认证与个人信息接口

该类接口主要服务用户端与管理端登录态建立和基础资料维护。当前项目的关键接口如下：

接口路径	方法	说明
`/api/register/`	POST	用户注册。
`/api/login/`	POST	普通用户登录。
`/api/logout/`	POST	用户登出。
`/api/token/refresh/`	POST	刷新认证令牌。
`/api/user/details/`	GET	获取当前用户详情。
`/api/user/update/`	PUT	更新用户资料。
`/api/user/avatar/`	PUT	更新用户头像。
`/api/password-reset/`	POST	发送密码重置验证码。
`/api/password-reset/confirm/`	POST	提交验证码并完成密码重置。
`/api/admin-login/`	POST	管理员登录。

从设计上看，这类接口构成所有业务接口的前置条件。后续在扩展统一资源与统一任务接口时，仍然需要依赖这一层提供用户身份、角色和组织上下文。

6.9 组织与成员接口

组织相关接口用于支撑组织入驻、组织审批、邀请码管理和组织权限维护。当前关键接口如下：

接口路径	方法	说明
`/api/organization/create/`	POST	提交组织申请。

接口路径	方法	说明
/api/organizations/create-directly/	POST	管理端直接创建组织。
/api/organization/applications/get_pending/	GET	获取待审批的组织申请列表。
/api/organization/applications/<app_id>/	GET	查看组织申请详情。
/api/organization/<app_id>/approve/	POST	审批通过组织申请。
/api/organization/<app_id>/reject/	POST	拒绝组织申请。
/api/organizations/	GET	获取组织列表。
/api/organization/<org_id>/	GET	获取组织详情。
/api/organization/<org_id>/delete/	DELETE	删除组织。
/api/organization/<org_id>/permission/	POST	更新组织角色权限。
/api/organization/<org_id>/invitation_codes/	GET	获取邀请码列表。
/api/organization/usage/	GET	查询组织检测次数使用情况。
/api/organization/recharge-uses/	POST	为用户或组织补充检测次数。

6.10 资源上传与资源管理接口

该类接口负责接收上传文件、读取文件详情、获取提取后的图像、维护文件标签和删除资源。当前实现仍以文件和图像资源为中心，后续论文与 Review 资源可在统一资源接口下复用同一接入逻辑。

接口路径	方法	说明
/api/upload/	POST	上传文件，支持图片、PDF、压缩包等。
/api/upload/<file_id>/	GET	获取单个上传文件详情。
/api/upload/<file_id>/extract_images/	GET	获取从文件中提取出的图像集合。
/api/upload/<file_id>/addTag/	POST	更新文件标签。
/api/upload/<file_id>/delete/	DELETE	删除上传文件。
/api/upload/get_all_file_images/<file_management_id>/	GET	获取某个文件下的全部图像。
/api/get_files/	GET	管理端获取文件列表。
/api/delete_image_upload/<image_id>/	DELETE	管理端删除单张图像资源。

6.11 检测任务与检测结果接口

检测接口是当前项目接口体系的核心。现有实现已经形成图像检测任务的完整闭环，包括任务提交、状态查询、结果列表、详情查询和报告下载。后续论文检测与 Review 检测建议继续沿用统一任务接口设计。

接口路径	方法	说明
/api/detection/submit/	POST	提交图像检测任务。
/api/detection/<image_id>/	GET	获取单张图像的检测结果。
/api/detection-task/<task_id>/status/	GET	用户端查询检测任务状态。
/api/get_detection_task_status/<task_id>/	GET	管理端查询检测任务状态。
/api/tasks/<task_id>/results/	GET	获取任务全部结果。
/api/tasks/<task_id>/fake_results/	GET	获取任务中判定为可疑的结果。
/api/tasks/<task_id>/normal_results/	GET	获取任务中判定为正常的结果。
/api/results/<result_id>/	GET	获取单条检测结果详情。
/api/results_image/<image_id>/	GET	通过图像查询检测结果。
/api/tasks_image/<image_id>/getdr/	GET	通过图像获取检测结果映射。
/api/tasks/<task_id>/report/	GET	下载任务级检测报告。
/api/tasks_image/<image_id>/report/	GET	下载图像级检测报告。
/api/detection-task-delete/<task_id>/	DELETE	删除检测任务。
/api/user-tasks/	GET	获取当前用户检测任务列表。
/api/task-summary/	GET	获取当前用户任务摘要。
/api/get-task-summary/	GET	获取任务汇总信息。

6.12 人工审核接口

人工审核接口构成自动检测之后的复核链路。该类接口覆盖审核申请、审核任务查看、审核结果查看、审核提交和人工审核报告生成。

接口路径	方法	说明
/api/create_review_task_with_admin_check/	POST	创建人工审核申请，等待管理员审批。
/api/get_request_completion_status/<task_id>/	GET	查询某检测任务对应审核申请的完成情况。
/api/get_request_detail/<reviewRequest_id>/	GET	获取审核申请详情。
/api/get_publisher_review_tasks/	GET	发布者查看本人发起的审核任务。
/api/get_reviewer_tasks/	GET	审核人员获取待处理审核任务列表。
/api/get-reviewer-request-detail/<reviewRequest_id>/	GET	审核人员查看审核申请详情。
/api/get_review_detail/<manual_review_id>/	GET	获取单个审核任务详情。
/api/post_review/<manual_review_id>/	POST	提交人工审核结果。
/api/get_img_review_all/	GET	获取审核任务下所有图像审核结果。
/api/get_image_review/	GET	获取指定图像的人工审核结果。
/api/manual-review/<review_id>/report/	GET	生成或下载人工审核报告。

接口路径	方法	说明
/api/manual-review/<review_request_id>/	GET	通过审核申请获取关联人工审核记录。
/api/publisher-dectectiontask-access/	GET	检查发布者是否有权访问某检测任务。
/api/reviewer-manualreview-access/	GET	检查审核人员是否有权访问某人工审核记录。
/api/publishers/<publisher_id>/reviewers/	GET	获取发布者可关联审核人员。
/api/get_all_reviewers/	GET	获取组织内审核人员列表。

6.13 通知与实时消息接口

通知接口分为持久化通知接口和实时推送接口两类。持久化通知负责通知列表和已读状态维护，实时推送接口负责检测完成等事件的在线通知。

接口路径	方法	说明
/api/notification/get/	GET	获取当前用户通知列表。
/api/notification/notify/	GET	获取当前用户未读通知数量。
/api/notification/set_as_read/	POST	将当前用户全部通知设为已读。
/api/notification/set_as_read/<notification_id>/	POST	将单条通知设为已读。
/api/notification/broadcast/	POST	管理端发送广播通知。
/ws/notifications/	WebSocket	实时推送通知与任务状态更新。

6.14 管理端治理与统计接口

管理端接口覆盖管理员资料、仪表盘统计、用户管理、文件管理、审核审批和日志查询等能力。该类接口支撑当前项目的治理能力，也是后续扩展论文检测日志、Review 检测日志和模型配置管理的入口。

接口路径	方法	说明
/api/admin/details/	GET	获取当前管理员详情。
/api/admin/details/<user_id>	GET	获取指定用户详情。
/api/admin_dashboard/	GET	获取管理端仪表盘汇总数据。
/api/dashboard/img_tag/	GET	图像标签统计。
/api/dashboard/top_publishers/	GET	发布者排行统计。
/api/dashboard/top_organizations/	GET	组织排行统计。
/api/dashboard/daily_active_users/	GET	日活用户统计。
/api/dashboard/daily_active_organizations/	GET	日活组织统计。
/api/dashboard/daily_task_count/	GET	每日检测任务统计。
/api/dashboard/daily_review_request_count/	GET	每日审核申请统计。

接口路径	方法	说明
/api/dashboard/daily_completed_manual_review_count/	GET	每日完成审核统计。
/api/dashboard/get_sub_method_distribution_by_tag/	GET	子检测方法分布统计。
/api/get_users/	GET	获取用户列表。
/api/create_user/	POST	管理端创建用户。
/api/update_user/<user_id>/	PUT	更新用户。
/api/delete_user/<user_id>/	DELETE	删除用户。
/api/create-admin/	POST	创建管理员。
/api/user_permission/<user_id>/	POST	更新用户权限。
/api/manage-associations/	POST	建立发布者与审核者关联。
/api/user_action_log/	GET	查询用户操作日志。
/api/user_action_log/<log_id>/	DELETE	删除日志。
/api/user_action_log/download/	GET	导出日志。
/api/get_reviewRequest/all/	GET	获取全部审核申请。
/api/get_reviewRequest/<reviewRequest_id>/	GET	获取审核申请详情。
/api/handle_reviewRequest/<reviewRequest_id>/	POST	管理员处理审核申请。
/api/get_review_request_detail/<manual_review_id>/	GET	查看人工审核执行详情。
/api/post_report/<post_id>/	POST	处理帖子举报。
/api//review-requests/<review_request_id>/delete/	DELETE	删除审核申请；当前代码路径定义带前导斜杠，建议后续修正。

6.15 内部接口设计

6.15.1 后端与异步任务系统接口

当前系统通过检测提交接口创建任务后，调用 Celery 执行异步检测。该类内部接口主要承担以下职责：

- 接收检测任务 ID、图像 ID 集合、检测参数和用户上下文；
- 在异步执行中更新任务状态、检测结果、报告生成状态和通知状态；
- 在任务完成后通过消息通道向 WebSocket 消费者推送结果。

6.15.2 后端与 AI 推理服务接口

当前项目的图像检测能力通过后端调用图像推理逻辑实现。后续论文检测和 Review 检测建议采用与图像检测一致的适配方式：

- 后端传入资源路径、文本内容或结构化输入；
- 推理层返回风险评分、分类结论、可视化结果或结构化分析结果；
- 后端统一完成结果落库、状态更新、报告生成和通知触发。

6.15.3 后端与文件存储接口

文件存储接口不直接对前端暴露，而由业务服务封装。主要职责包括：

- 保存上传文件、提取图像、检测中间产物和报告文件；
- 提供受控读取能力，供结果查询和报告下载接口调用；
- 在删除任务或删除资源时执行对应文件清理。

6.15.4 后端与消息通道接口

后端通过 Django Channels 将状态变化推送到 WebSocket 通道。内部消息接口应保证：

- 推送事件与数据库通知记录保持一致；
- 用户上线和离线不影响通知持久化；
- 前端可基于同一消息体刷新未读数、任务状态或详情页数据。

6.16 接口安全设计

接口安全主要包括以下方面：

- 需要登录的接口必须通过认证令牌识别当前用户；
- 用户只能访问本人资源、本人检测任务、本人通知和权限范围内的审核记录；
- 管理端接口必须校验管理员身份和管理权限；
- 文件下载接口必须校验资源归属和访问权限，避免直接暴露物理路径；
- 审核、权限调整、组织审批、日志导出等高风险接口必须记录审计日志。

6.17 接口演进方向

从当前项目代码看，接口仍然明显偏向图像检测场景。后续接口演进建议如下：

- 引入统一资源接口，将图像、论文和 Review 统一纳入资源上传和资源查询入口；
- 引入统一检测任务接口，使图像检测、论文检测和 Review 检测共用任务提交和状态查询协议；
- 保留差异化结果接口，分别承载图像可疑区域、论文段落概率和 Review 模板化倾向等差异结果；
- 将论文检测日志、Review 检测日志和模型配置接口纳入管理端统一治理接口空间；
- 对现有路径命名、重复路由和个别异常路径进行收敛，提升整体接口规范性。

6.18 本章小结

本章从概要设计角度给出了系统接口体系的总体设计。当前项目已经形成以用户认证、资源上传、图像检测、人工审核、通知和管理端治理为主体的接口闭环；后续论文检测与 Review 检测能力可在统一资源、统一任务和差异化结果接口框架下继续扩展。接口设计的核心目标不是增加更多零散接口，而是在现有工程基础上形成可持续演进的统一接口体系。

7 关键流程设计

本章从系统运行角度说明用户提交资源后，平台如何完成资源登记、任务创建、异步检测、人工审核、报告输出与状态通知。与前几章的静态模块和数据设计不同，本章强调对象在时间维度上的流转顺序，以及不同模块之间的协作边界。

需要说明的是：当前代码主干已经完整实现图像上传、图像检测、人工审核和通知闭环；论文检测与 Review 检测尚未形成独立生产链路，因此本章对这两部分采用“迭代目标流程”的方式描述，并明确其与现有图像检测流程的衔接点。

7.1 资源上传与检测流程

资源上传与检测发起流程是整个平台的入口流程。当前实现中，系统支持用户上传单张图像、PDF 文档以及压缩包三类输入，其中 PDF 和压缩包会在上传阶段被解析为可检测的图像集合。随后用户从已登记资源中选择图像发起检测，系统完成任务创建、额度校验和异步调度。

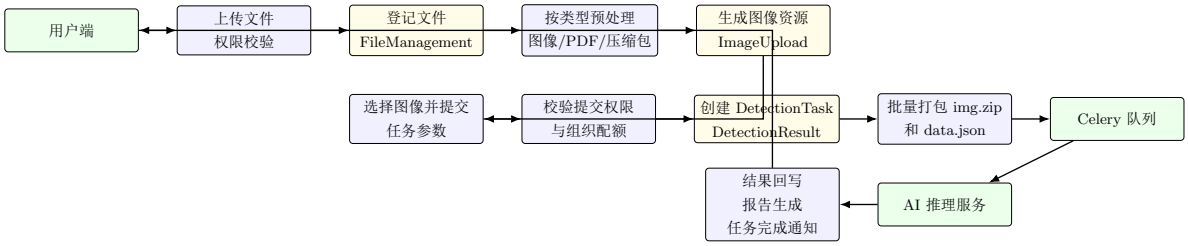

图 5: 资源上传与检测发起流程图

7.1.1 流程输入

- 用户登录态、用户角色和权限信息；
- 原始上传资源，包括图像文件、PDF 文件或压缩包；
- 用户选择的检测图片集合、任务名称和检测参数。

7.1.2 处理步骤

1. 用户上传文件后，后端首先校验其上传权限，随后在 `FileManagement` 中登记文件元数据；
2. 系统将原始文件写入媒体存储目录，并根据文件类型进入不同的预处理分支；
3. 若上传对象为单张图像，则直接创建对应的 `ImageUpload` 记录；若为 PDF，则提取其中图像页并批量生成 `ImageUpload`；若为压缩包，则依次处理其中的图像和 PDF；
4. 上传完成后，系统写入上传日志，前端通过文件详情接口或图像列表接口获取当前可检测图像集合；
5. 用户在前端选择待检测图像并提交任务，后端校验提交权限、组织配额和检测参数；
6. 系统创建 `DetectionTask`，并为选中图像逐一建立 `DetectionResult` 初始记录，状态置为处理中；
7. 后端按批次将图像打包为临时压缩文件和参数文件，并将批次任务提交给 Celery 队列等待异步执行。

7.1.3 流程输出

- 已登记的文件记录和图像记录；
- 处于待处理或处理中状态的检测任务；
- 后续异步检测阶段所需的批处理上下文。

7.1.4 关键控制点

- 上传阶段必须先进行权限判定，避免无上传权限用户占用存储空间；
- 检测提交前需完成组织额度校验，并在校验通过后立即扣减，防止并发下超额提交；
- 资源预处理和任务提交之间通过数据库对象衔接，避免前端直接依赖临时文件路径。

7.2 图像检测流程

图像检测流程描述的是异步任务真正执行的主链路。当前项目采用“批量发送到 AI 服务，再按单图并行后处理，最后任务级汇总收尾”的三阶段模式，兼顾了远程推理服务的调用效率和结果落库的并发能力。

7.2.1 流程阶段划分

1. **批量获取阶段：** Celery 的 `fetch_batch` 任务读取批次目录中的 `img.zip` 和参数文件，向远程 AI 推理服务发起整批请求；
2. **单图后处理阶段：** 对每个 `DetectionResult` 调用 `process_single_result`，完成可视化图片保存、主结果更新、子方法结果写入和图像状态更新；
3. **任务收尾阶段：** 当同一检测任务下所有图片的结果均完成后，执行 `finalize_task`，统一回写任务完成时间、生成检测报告并向用户推送完成通知。

7.2.2 详细处理步骤

1. 异步任务启动后，将 `DetectionTask` 状态更新为进行中，并将本批次对应的 `DetectionResult` 状态标记为处理中；
2. 系统调用远程 AI 推理服务获取整批检测结果，必要时执行重连与重试；
3. 对批次中的每张图，系统分别解析总体真假判定、置信度、LLM 文本说明、ELA 可视化、EXIF 判定和各子方法掩码结果；
4. 后处理任务将可视化图像保存到媒体目录，并更新 `DetectionResult`、`SubDetectionResult` 与 `ImageUpload`；
5. 系统统计同一任务下已完成结果数量，若全部完成，则将任务状态更新为已完成并生成任务级 PDF 报告；
6. 任务完成后，通过 WebSocket 群组向用户发送检测完成通知，前端据此刷新任务状态或提示用户查看结果。

7.2.3 流程输出

- 图像级检测结论、可视化结果和子方法输出；
- 已完成状态的检测任务和可下载的任务级报告；
- 面向用户端的任务完成通知。

7.2.4 异常与恢复

- 当 AI 推理服务不可达或返回数量异常时，批处理任务执行重试与重连逻辑；
- 若个别图片后处理失败，不应提前将整个任务置为完成状态；
- 若报告文件尚未生成，则报告下载接口返回“正在生成”而不是直接报错。

7.3 论文检测流程

论文检测属于本轮迭代的目标能力。当前代码尚未形成独立的论文解析、段落切分和论文级结果实体，但根据需求文档和现有图像链路，可以将论文检测流程设计为“论文资源登记 -¿ 文本/图像双通道解析 -¿ 段落级分析 -¿ 论文级结果聚合”的模式。

7.3.1 目标流程

1. 用户上传论文文件后，系统先完成文件登记和基础格式校验；
2. 解析模块提取论文正文、章节结构、元信息和内嵌图像，并建立论文资源与图像资源的关联；
3. 文本分析模块对论文进行段落切分，执行 AIGC 倾向分析、事实性说明生成和异常段落标记；
4. 若论文中包含图像资源，则可选择复用现有图像检测链路，对论文插图执行独立鉴伪；
5. 聚合模块将文本检测结果、插图检测结果和整体风险评分整合为论文级结论；
6. 报告模块输出论文检测报告，并在用户端展示高风险段落、对应说明和插图结果摘要。

7.3.2 与当前实现的衔接点

- 资源登记仍可复用现有文件上传和媒体存储机制；
- 任务状态流转、异步调度、报告生成和通知机制可复用当前图像检测任务框架；
- 需要新增的主要能力集中在论文解析、段落级结果建模和论文报告聚合。

7.3.3 设计关注点

- 论文检测输出不应只有单一真假结论，而应保留段落级风险与解释信息；
- 论文文本链路和论文插图链路需要统一汇总口径，避免生成两套割裂的报告；
- 论文检测属于迭代能力，本阶段流程设计应强调扩展接口和复用现有基础设施。

7.4 Review检测流程

Review 检测同样属于迭代目标能力。该流程面向期刊审稿意见、同行评审文本或内部评阅意见，目标是识别模板化倾向、AI 生成倾向和文本表达异常。

7.4.1 目标流程

1. 用户提交 Review 文本或上传包含 Review 内容的文件；
2. 系统完成文本清洗、编码统一、结构化分段和长度校验；
3. 检测模块分别执行模板化相似性分析、AI 生成倾向分析、术语密度与意见覆盖度分析；
4. 聚合模块根据多项指标生成 Review 风险评分与判定说明；
5. 结果层向前端返回总体结论、风险证据和可疑片段高亮信息；
6. 报告层生成可归档的 Review 检测结果摘要，供期刊管理或后续人工复核使用。

7.4.2 与当前实现的衔接点

- 任务创建、异步执行、通知和日志记录可沿用现有检测任务框架；
- 需要新增 Review 资源表示、文本分析器和 Review 结果展示组件；
- 若 Review 检测结论需要人工复核，可复用当前人工审核模块的审批与指派思想。

7.4.3 设计关注点

- Review 检测更强调“风险提示”而非简单真假二分，需要保留证据型输出；
- 结果展示应适配文本高亮和说明片段，不宜直接套用图像检测结果页；
- 后续若引入模型管理，应记录 Review 检测所用模型版本与规则版本。

7.5 人工审核流程

人工审核流程用于处理用户对自动检测结果存在异议、或者系统要求进一步专家复核的场景。当前代码已经形成“发布者申请 -¿ 管理员审批 -¿ 审核员执行 -¿ 审核结果回传”的完整链路。

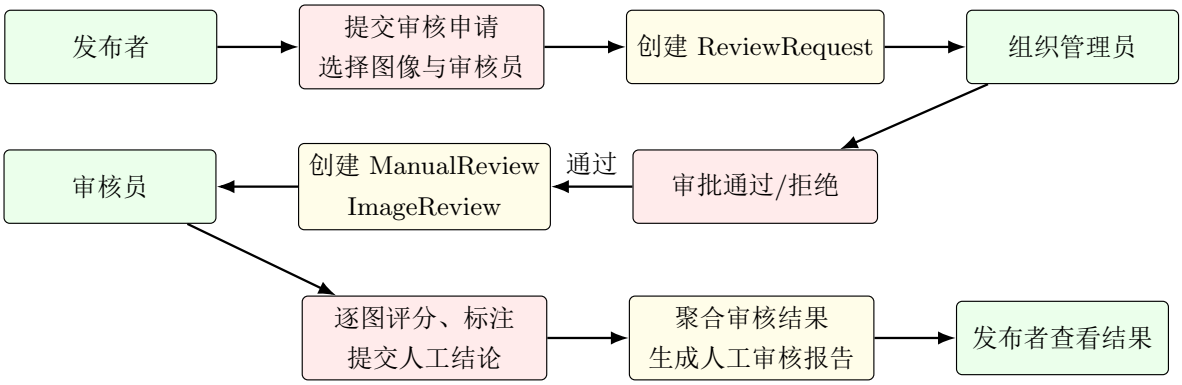

图 6: 人工审核流程图

7.5.1 处理步骤

1. 发布者在检测结果页选择若干图片并提交人工审核申请，系统创建 `ReviewRequest`，记录申请理由、关联图片和指定审核员；
2. 系统向组织管理员发送待审批提醒，当前实现中该提醒通过邮件发送；
3. 管理员审核申请后，若拒绝，则更新审核请求状态并结束流程；若通过，则为每个审核员创建一条 `ManualReview`，并为每张图初始化 `ImageReview`；
4. 系统向被分派审核员发送站内通知，审核员在前端查看待办任务并逐图填写评分、理由、标注点和人工结论；
5. 审核员提交后，系统保存逐图审核数据并更新人工审核状态；
6. 当发布者或管理端查看审核结果时，系统聚合所有已完成审核记录，展示不同审核员的判定结果，并在需要时生成人工审核报告。

7.5.2 流程输出

- 审核申请记录、审核执行记录和逐图审核结果；
- 审核员侧待办通知和发布者侧审核完成结果；
- 可下载的人工审核 PDF 报告。

7.5.3 关键控制点

- 审核申请仅允许具备发布权限且角色为发布者的用户发起；
- 审核员范围需要受组织和角色约束，避免跨组织指派；
- 管理员审批和审核员执行属于两个独立状态层，流程设计中必须分开建模；
- 审核记录应保留逐图评分与理由，保证人工结论具备复核依据。

7.6 通知与状态同步流程

通知与状态同步流程用于保证用户在检测和审核过程中能够及时获取任务变化。当前实现采用“数据库持久化通知 + WebSocket 实时推送 + REST 查询补偿”的组合方案。

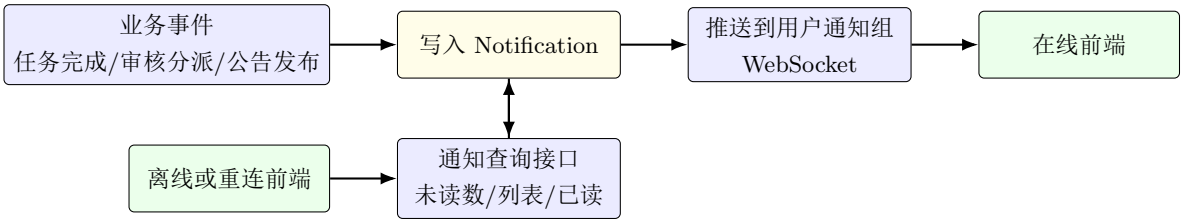

图 7: 通知与状态同步流程图

7.6.1 处理步骤

1. 业务模块在任务完成、人工审核分派、管理员审批或公告发送时创建 `Notification` 记录；
2. 在线用户通过带令牌参数的 WebSocket 连接加入对应的用户通知组；
3. 当检测任务收尾阶段完成后，系统向用户对应的通知组推送实时消息，前端接收后刷新任务状态提示；
4. 若用户不在线，则通知记录保留在数据库中，用户下次进入系统后可通过通知接口拉取未读消息；
5. 用户可通过接口将全部通知或单条通知标记为已读，管理端则可以通过广播接口向所有普通用户发布系统公告。

7.6.2 流程特点

- WebSocket 负责降低用户等待期间的轮询压力，但系统不依赖 WebSocket 作为唯一通知通路；
- 通知记录落库后，即使实时推送失败，用户仍可通过接口补拉消息；
- 当前实现的检测完成通知主要面向用户级群组，而不是面向单任务群组统一广播。

7.7 流程图说明

本章流程图采用“业务主线优先、异常分支从简”的表达策略。图中保留了对概要设计最关键的角色、模块和状态转换，不展开到接口参数或数据库字段级别。各图与正文的对应关系如下：

- 图 7-1 对应资源上传与检测发起流程，体现资源登记、任务创建和异步调度的衔接；
- 图 7-2 对应人工审核流程，体现发布者、管理员和审核员三方协作关系；
- 图 7-3 对应通知与状态同步流程，体现数据库通知、实时推送和接口补偿三者的协同。

后续进入详细设计阶段时，可在本章基础上继续细化为接口时序图、状态机图和异常处理分支图，从而为实现与测试提供更直接的依据。

8 运行与部署设计

8.1 部署结构

系统部署上至少包含以下节点：

- Web前端部署节点；
- Django后端服务节点；
- Redis与异步任务处理节点；
- AI推理服务节点；
- MySQL数据库节点；
- 文件存储目录或对象存储节点。

8.2 运行机制

用户请求首先到达前端和后端，短请求由后端同步处理，长时检测任务通过异步队列提交到任务系统，再由AI推理服务执行，最终检测结果、日志和报告回写数据库与文件存储，并通过轮询或消息机制反馈到前端。

8.3 部署约束

- AI推理服务对运行环境、模型文件和GPU资源有依赖；
- 异步任务系统依赖消息中间件与稳定的任务状态存储；
- 文件上传、结果图片和PDF报告需要统一的存储路径管理；
- 生产部署时需明确静态资源、媒体文件、WebSocket与API代理配置。

9 质量属性设计

9.1 性能设计

系统需要支持批量上传、异步检测、状态同步和报告生成等场景，避免长时任务阻塞常规交互请求。

9.2 安全设计

系统需采用身份认证、角色权限控制、敏感数据保护、文件上传校验和关键操作审计等机制保障运行安全。

9.3 可维护性设计

系统应通过模块化拆分、统一抽象、接口适配和日志留痕提升维护效率，并降低新模型接入或新资源类型扩展带来的改动范围。

9.4 可扩展性设计

系统在设计上需要支持：

- 新增检测对象；
- 新增检测模型；
- 扩展报告内容；
- 扩展管理统计维度；
- 扩展人工审核规则。

9.5 可靠性设计

系统需具备异常提示、任务重试、日志追踪和必要的数据恢复能力，以降低网络异常、模型失败或服务中断对业务流程的影响。

10 设计约束与待定问题

10.1 当前约束

- 现有实现以学术图像检测为核心，论文与Review能力仍需扩展；
- 当前数据模型与接口设计中存在图像中心化特征；
- AI推理链路与后端任务链路的耦合仍需在后续设计中进一步梳理；
- 部分管理能力和模型配置能力尚未形成统一抽象。

10.2 待定问题

- 统一资源抽象与现有图像表结构的兼容迁移方案；
- 论文检测与Review检测的结果数据结构；
- 综合报告中多类型资源结果的整合方式；
- 模型管理配置在管理端中的可视化表达方式；
- 论文与Review专用日志的存储粒度和查询维度。

10.3 后续工作

后续将基于本概要设计继续细化详细设计，包括数据库设计、接口字段设计、模块内部类与函数设计、关键流程图、异常处理策略和测试设计。